REPUBLIQUE DU CAMEROUN
* * * * * * * * * *
PAIX — TRAVAIL — PATRIE
* * * * * * * * * *
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

REPUBLIC OF CAMEROON
* * * * * * * * * *
PEACE – WORK – FATHERLAND
* * * * * * * * * *
MINISTRY OF HIGHER EDUCATION

INTERNSHIP REPORT

DEVELOPMENT OF A WEB APPLICATION VULNERABILITY SCANNER

crawling, security headers, TLS, redirects, URL-risk analysis and PDF reporting

Internship carried out at IWOMI TECHNOLOGIES, Douala, Bonanjo

From July 1 to August 31, 2026

Field: BACHELOR OF ENGINEERING (COMPUTER ENGINEERING).

Written and presented by:

MBA SIMON ASI.

Student Registration No: **IUC24E0080002**.

Under the supervision of:

Academic Supervisor:	Professional Supervisor:
FOMAZOU TCHINDA BRICE	**Mr. KEN**

ACADEMIC YEAR 2026-2027

REPUBLIQUE DU CAMEROUN
* * * * * * * * * *
PAIX — TRAVAIL — PATRIE
* * * * * * * * * *
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

REPUBLIC OF CAMEROON
* * * * * * * * * *
PEACE – WORK – FATHERLAND
* * * * * * * * * *
MINISTRY OF HIGHER EDUCATION

INTERNSHIP REPORT

DEVELOPMENT OF A WEB APPLICATION VULNERABILITY SCANNER

crawling, security headers, TLS, redirects, URL-risk analysis and PDF reporting

Internship carried out at IWOMI TECHNOLOGIES, Douala, Bonanjo

From July 1 to August 31, 2026

Field: BACHELOR OF ENGINEERING (COMPUTER ENGINEERING).

Written and presented by:

MBA SIMON ASI.

Student Registration No: **IUC24E0080002**.

Under the supervision of:

Academic Supervisor:	Professional Supervisor:
FOMAZOU TCHINDA BRICE	**Mr. KEN**

ACADEMIC YEAR 2026-2027

CERTIFICATION

This is to certify that the internship report entitled **"Development of a Web Application Vulnerability Scanner"** is the original work carried out by **MBA SIMON ASI** at IWOMI Technologies, Douala, from July 1 to August 31, 2026, in partial fulfilment of the requirements for the award of the Bachelor of Engineering (B.Eng.) degree in Computer Engineering at the School of Engineering and Applied Sciences (SEAS) of the Institut Universitaire de la Côte (IUC).

APPROVED BY:

ACADEMIC SUPERVISOR	**HEAD OF DEPARTMENT**
FOMAZOU TCHINDA BRICE	SIGNATURE……
SIGNATURE……	DATE……
DATE……	

DEDICATION

To my family, for their love, their sacrifices and their constant support throughout my studies.

ACKNOWLEDGEMENTS

This internship and this report would not have been possible without the help of many people. I am deeply grateful to all of them.

First of all, I thank Almighty God for the life, the health and the strength granted to me throughout this internship. I also express my sincere gratitude to:

- **Mr. GUIMEZAP Paul**, Founder and President of the Institut Universitaire de la Côte (IUC), for his vision and for the environment created for students to grow and innovate;
- **The Director of the School of Engineering and Applied Sciences (SEAS) and the Head of the Computer Engineering Department**, for the quality of our training and their constant encouragement;
- **FOMAZOU TCHINDA Brice**, my academic supervisor, for the availability, the advice and the careful review of this work;
- **Mr. KEN**, my professional supervisor at IWOMI Technologies, for welcoming me into the company, for the assignments entrusted to me and for the guidance given throughout these nine weeks;
- **The management and the whole team of IWOMI Technologies**, for their warm welcome, for the time spent explaining their products and their working methods, and for their feedback on my work;
- **All the lecturers of the IUC**, for the knowledge and the method they passed on to us;
- **My family and my friends**, for their moral and material support at every stage of this work.

TABLE OF CONTENTS

CERTIFICATION ..... **i**
DEDICATION ..... **ii**
ACKNOWLEDGEMENTS ..... **iii**
LIST OF FIGURES ..... **vii**
LIST OF TABLES ..... **viii**
LIST OF ACRONYMS ..... **ix**
ABSTRACT ..... **x**
RÉSUMÉ ..... **xi**
GENERAL INTRODUCTION **1**
General context of the study 1
Definition of key concepts 1
Problem statement 1
Aim and objectives of the study 2
Methodology 2
Organization of the report 2
PART ONE: CONCEPTUAL FRAMEWORK **3**
CHAPTER 1 PRESENTATION OF THE COMPANY AND ACTIVITIES CARRIED OUT DURING THE INTERNSHIP **4**
Introduction 4
1.1 Presentation of IWOMI Technologies 4
1.1.1 History and legal identity 4
1.1.2 Mission and vision 4
1.1.3 Products and services 5
1.1.4 Organisation 5
1.2 Activities carried out during the internship 6
1.2.1 First assignment: reproduction of the company website 7
1.2.2 Main project: the web application vulnerability scanner 8
1.3 Choice and justification of the topic 8
Conclusion 8
CHAPTER 2 OVERVIEW OF THE TOPIC STUDIED (LITERATURE REVIEW) **9**
Introduction 9
2.1 Web application security 9
2.1.1 Objectives of security 9
2.1.2 The OWASP Top 10 (2021) 9
2.2 Security controls checked by the scanner 10
2.2.1 HTTP security headers 10
2.2.2 Cookies 10
2.2.3 Transport security (HTTPS and TLS) 10
2.2.4 Redirects 11
2.2.5 Phishing and suspicious URLs 11
2.3 Existing tools for web security assessment 11
2.4 Technologies of the solution 12
2.5 Development process 12
2.6 Legal and ethical framework 12
Conclusion 12

PART TWO: EMPIRICAL STUDY **14**
CHAPTER 3 DIAGNOSTIC OF THE EXISTING SYSTEM AND REQUIREMENTS SPECIFICATION **15**
Introduction 15
3.1 The existing security assessment process 15
3.1.1 Description 15
3.1.2 Observation of a manual assessment 15
3.1.3 Problems identified 16
3.1.4 Strengths to keep 16
3.2 Requirements specification 16
3.2.1 Actors 16
3.2.2 Functional requirements 16
3.2.3 Non-functional requirements 17
3.3 Feasibility and risks 17
Conclusion 18
CHAPTER 4 PROPOSED SOLUTION AND IMPLEMENTATION **19**
Introduction 19
4.1 Presentation of the scanner 19
4.1.1 Evolution of the project 19
4.2 Design 20
4.2.1 Use case diagram 20
4.2.2 Architecture 20
4.2.3 Scan pipeline 21
4.2.4 Sequence diagram 21
4.2.5 Data model 21
4.2.6 Code organisation 22
4.3 Implementation 22
4.3.1 Target validation and authorization 22
4.3.2 Crawler 22
4.3.3 Passive checks 22
4.3.4 External tools 23
4.3.5 Correlation, categories and score 23
4.3.6 Background execution and persistence 23
4.3.7 Defects found and corrected 24
4.4 Presentation of the interfaces 24
4.5 Tests and validation 28
4.5.1 Laboratory website 28
4.5.2 Automated tests 29
4.5.3 Results on the laboratory site 29
4.5.4 Performance 30
4.5.5 Coverage of the requirements 30
4.6 Installation and configuration 30
4.7 Cost estimate of the solution 30
4.8 Limits of the current version 31
Conclusion 31
GENERAL CONCLUSION **32**
Summary of the work 32
Methodology and solution 32

Difficulties encountered....32
Perspectives....32
BIBLIOGRAPHY AND WEBOGRAPHY....33
Bibliography....33
Webography....33
APPENDIX....35
Appendix 1: List of the automated tests....35
Appendix 2: Extract of the scan function....36
Appendix 3: Additional screenshots....37

LIST OF FIGURES

Figure 1: Main products of IWOMI Technologies (source: author)..... 5
Figure 2: Planning of the internship (nine weeks) (source: author)..... 6
Figure 3: Reproduction of the IWOMI website: navigation bar and banner (source: author)..... 7
Figure 4: Reproduction of the IWOMI website: “Who are we?” section (source: author)..... 8
Figure 5: Existing manual assessment process and its problems (source: author)..... 15
Figure 6: Use case diagram of the scanner (source: author)..... 20
Figure 7: Architecture of the scanner (source: author)..... 20
Figure 8: Stages of a scan (source: author)..... 21
Figure 9: Sequence diagram “start a scan and follow its progress” (source: author)..... 21
Figure 10: Data model (SQLite) (source: author)..... 22
Figure 11: Layers and folders of the project (source: author)..... 22
Figure 12: New scan page with the mandatory authorization and the recent scans (source: author)..... 24
Figure 13: Progress page during a scan (source: author)..... 25
Figure 14: Results of the weak laboratory site: grade C, 72/100 (source: author)..... 25
Figure 15: Detail of a finding with its evidence and recommendation (source: author)..... 26
Figure 16: Results of the hardened laboratory site: grade A, 92/100 (source: author)..... 26
Figure 17: URL risk check of a suspicious link: high risk (source: author)..... 27
Figure 18: History of the scans (source: author)..... 27
Figure 19: First page of the PDF report generated for the weak laboratory site (source: author)..... 28
Figure 20: Automated tests per area (21/21 passed) (source: author)..... 29
Figure 21: Findings per severity on the two variants of the laboratory site (source: author)..... 29
Figure 22: URL risk check of the company domain: low risk (heuristic check) (source: author)..... 37
Figure 23: Complete results page of the weak laboratory site (source: author)..... 38

LIST OF TABLES

Table 1: Identity card of IWOMI Technologies....4
Table 2: Products of IWOMI Technologies....5
Table 3: Organisation of IWOMI Technologies as observed during the internship....6
Table 4: Activities carried out week by week....6
Table 5: The OWASP Top 10 (2021) and what the scanner checks....9
Table 6: Security headers checked by the scanner....10
Table 7: Comparison of existing web security tools....11
Table 8: Observation of one manual assessment....15
Table 9: Functional requirements....16
Table 10: Non-functional requirements....17
Table 11: Main risks of the project and their mitigation....18
Table 12: Versions of the scanner....19
Table 13: Computation of the security score....23
Table 14: Defects found during the internship and their correction....24
Table 15: Results of the scanner on the two variants of the laboratory site....29
Table 16: Measured performance (median of repeated runs, local machine)....30
Table 17: Cost estimate of the solution (indicative)....30
Table 18: Automated tests of the scanner....35

LIST OF ACRONYMS

Acronym	Meaning
API	Application Programming Interface
ATM	Automated Teller Machine
CSP	Content Security Policy
CSS	Cascading Style Sheets
CVE	Common Vulnerabilities and Exposures
DAST	Dynamic Application Security Testing
FCFA	Franc de la Coopération Financière en Afrique centrale
FR / NFR	Functional / Non-Functional Requirement
HSTS	HTTP Strict Transport Security
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IDN	Internationalized Domain Name
IUC	Institut Universitaire de la Côte
JSON	JavaScript Object Notation
MIME	Multipurpose Internet Mail Extensions
MoSCoW	Must, Should, Could, Won't (prioritisation method)
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
SEAS	School of Engineering and Applied Sciences
SQL	Structured Query Language
TLD	Top-Level Domain
TLS / SSL	Transport Layer Security / Secure Sockets Layer
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
XSS	Cross-Site Scripting
ZAP	Zed Attack Proxy (OWASP ZAP)

ABSTRACT

This report presents the work carried out during my internship at **IWOMI Technologies**, a Douala-based fintech company that builds digital banking solutions for banks, microfinance institutions and consumers, from July 1 to August 31, 2026, as part of the Bachelor of Engineering programme in Computer Engineering at the Institut Universitaire de la Côte (IUC).

After a first familiarisation assignment on the company website, my work focused on a security need: checking a website by hand meant installing and learning several tools (OWASP ZAP, Nikto, Nmap, the browser developer tools), running each one separately, and then comparing reports written in different formats, with no history and no overall score. For a company whose clients are banks, this manual work is slow and hard to repeat.

To answer this problem I designed and built a **Web Application Vulnerability Scanner** in Python with the Flask framework. After the user confirms that they are authorized to test the target, the scanner crawls the pages and forms of the same host, checks the browser security headers and the cookie attributes, the TLS configuration and the redirect chains, analyses the risk of the URL with phishing heuristics, and can call OWASP ZAP, Nikto and Nmap when they are installed. All findings are deduplicated, classified by OWASP Top 10 (2021) category, turned into an explainable score from 0 to 100 with a grade from A to F, stored in SQLite and exported as a PDF report.

The project followed an iterative and incremental process. The final version is validated by **21 automated tests (21 passed)** run against a local laboratory website built in two variants, weak and hardened: the scanner gives the weak variant the grade C (72/100, 8 actionable findings) and the hardened variant the grade A (92/100), and a complete passive scan of the laboratory site takes about 50 ms.

Keywords: web application security, vulnerability scanner, OWASP Top 10, security headers, TLS, phishing detection, Python, Flask, SQLite, Cameroon.

RÉSUMÉ

Ce rapport présente le travail réalisé pendant mon stage chez **IWOMI Technologies**, fintech basée à Douala qui conçoit des solutions bancaires numériques pour les banques, les établissements de microfinance et les particuliers, du 1er juillet au 31 août 2026, dans le cadre du Bachelor of Engineering en Computer Engineering de l'Institut Universitaire de la Côte (IUC).

Après une première tâche de familiarisation sur le site web de l'entreprise, mon travail a porté sur un besoin de sécurité : vérifier un site à la main obligeait à installer et apprendre plusieurs outils (OWASP ZAP, Nikto, Nmap, outils de développement du navigateur), à les lancer séparément, puis à comparer des rapports de formats différents, sans historique ni note globale. Pour une entreprise dont les clients sont des banques, ce travail manuel est lent et difficile à répéter.

Pour répondre à ce problème, j'ai conçu et développé un **scanner de vulnérabilités d'applications web** en Python avec le framework Flask. Après confirmation par l'utilisateur qu'il est autorisé à tester la cible, le scanner parcourt les pages et formulaires du même hôte, vérifie les en-têtes de sécurité et les attributs des cookies, la configuration TLS et les chaînes de redirection, évalue le risque de l'URL par des heuristiques anti-hameçonnage, et peut appeler OWASP ZAP, Nikto et Nmap lorsqu'ils sont installés. Les résultats sont dédoublonnés, classés selon l'OWASP Top 10 (2021), traduits en une note explicable de 0 à 100 et une lettre de A à F, enregistrés dans SQLite et exportés en rapport PDF.

Le projet a suivi une démarche itérative et incrémentale. La version finale est validée par **21 tests automatisés (21 réussis)** exécutés sur un site de laboratoire local en deux variantes, vulnérable et renforcée : le scanner attribue la note C (72/100, 8 constats à corriger) à la première et la note A (92/100) à la seconde, et un scan passif complet du site de laboratoire dure environ 50 ms.

Mots-clés : sécurité des applications web, scanner de vulnérabilités, OWASP Top 10, en-têtes de sécurité, TLS, détection de l'hameçonnage, Python, Flask, SQLite, Cameroun.

GENERAL INTRODUCTION

General context of the study

Banking in Cameroon is becoming digital: mobile banking applications, online loan requests, transfers to mobile money accounts and web portals for customers and partners are now part of the daily offer of banks and microfinance institutions. Each of these services is exposed on the Internet through a website or a web application, and each one can be attacked. The OWASP Top 10, the reference list of the most critical web application security risks, shows that a large share of these risks comes not from complex attacks but from simple configuration mistakes: a missing security header, a cookie readable by scripts, a site still served over HTTP, an unnecessary redirect chain.

IWOMI Technologies, the host company of this internship, builds digital banking solutions for financial institutions. For such a company, the image and the security of web interfaces are two sides of the same trust. My internship therefore focused on building a tool that helps check the security of a website quickly, in a repeatable way, and only on targets that one is authorized to test.

Definition of key concepts

- **Vulnerability**: a weakness in an application, its configuration or its environment that an attacker could exploit to harm confidentiality, integrity or availability.
- **Web application vulnerability scanner**: a tool that sends requests to a web application and analyses the responses in order to detect possible weaknesses. Its findings are indications that must be interpreted; a scan alone never proves that a site is secure.
- **Passive check / active scan**: a passive check only reads normal responses (headers, cookies, certificate, redirects); an active scan sends crafted requests (attack payloads) and must only be run with explicit permission.
- **Phishing**: an attack that imitates a trusted site or message to steal credentials or money, often through a misleading URL.

Problem statement

At the start of the internship, a security check of a website was done by hand. The analyst opened the browser developer tools to read the headers and cookies, installed and configured OWASP ZAP, then Nikto and Nmap, ran each tool separately and obtained results in three or four different formats (HTML report, terminal text, screenshots). Nothing was kept from one check to the next, there was no overall score to compare two sites or two versions of a site, and the question “am I authorized to test this target?” depended only on the memory of the person. The central question of this work is therefore: **how can the security check of a website be made faster, consistent and traceable, while remaining limited to authorized targets?**

Aim and objectives of the study

The general aim is to apply computer engineering knowledge to practical web development and web security tasks within IWOMI Technologies. The specific objectives are:

- ➢ to understand the activities and products of IWOMI Technologies and the security needs of its web interfaces;
- ➢ to learn how the usual security tools (OWASP ZAP, Nikto, Nmap) work and to carry out a manual assessment of an authorized website;
- ➢ to analyse the manual process, list its problems and write the requirements of a tool that improves it;
- ➢ to design and build a web application in Python/Flask that gathers the checks, classifies the findings by OWASP Top 10 category, computes a score and produces a PDF report;
- ➢ to validate the tool with automated tests on a local laboratory website, measure its performance and state its limits honestly.

Methodology

The work followed the software development life cycle with an **iterative and incremental model**: observation of the existing practice and documentary research, requirements specification with the MoSCoW method, design with UML diagrams, then three increments (core checks, interface and external tools, persistence/score/tests), each one tested before the next. Every scan in this project was run on targets I was authorized to test: the authorized website indicated by my supervisor during the manual assessment, and a local laboratory website built for the purpose.

Organization of the report

Part One presents the conceptual framework: Chapter 1 introduces IWOMI Technologies, the activities carried out week by week and the choice of the topic; Chapter 2 reviews the concepts and tools of web application security. **Part Two** presents the empirical study: Chapter 3 diagnoses the existing manual process and specifies the requirements; Chapter 4 presents the proposed solution, its implementation, its tests and its cost. A general conclusion, the bibliography and the appendices close the report.

PART ONE:
CONCEPTUAL FRAMEWORK

CHAPTER 1
PRESENTATION OF THE COMPANY AND ACTIVITIES CARRIED OUT DURING THE INTERNSHIP

Introduction

This chapter presents the host company, IWOMI Technologies: its history, its activities, its products and its organisation. It then describes the activities I carried out during the nine weeks of the internship and explains the choice of the topic.

1.1 Presentation of IWOMI Technologies

1.1.1 History and legal identity

IWOMI Technologies is a Cameroonian fintech company founded in **2015** and based in **Bonanjo, Douala**. It was founded by Mr. Collins Fomba Ken, an engineer with more than fifteen years of experience in the implementation of banking software, who leads it as managing director. The company started by automating regulatory compliance reporting for banks and microfinance institutions, then extended its offer to digital banking channels and consumer payment applications.

In 2018, the first cash-deposit ATM in Cameroon, opened by Commercial Bank of Cameroon (CBC) at its Kribi branch, relied on an application designed by IWOMI Technologies. In February 2025, the pan-African accelerator The Baobab Network announced an investment in the company, presented as a Cameroonian fintech transforming the African banking sector.

Table 1: *Identity card of IWOMI Technologies*

Item	Information
Company name	IWOMI Technologies
Sector	Financial technology (fintech) – software for banks, microfinance institutions and consumers
Year of creation	2015
Head office	Bonanjo, Douala, Littoral Region, Cameroon
Founder and managing director	Mr. Collins Fomba Ken
Main clients	Banks and microfinance institutions in Cameroon and the CEMAC region; individual users of its consumer applications
Notable reference	Application of the first cash-deposit ATM in Cameroon (CBC, Kribi, 2018)

1.1.2 Mission and vision

The mission of IWOMI Technologies is to develop and integrate solutions that improve **financial inclusion** for African financial institutions and consumers: helping banks and microfinance institutions offer digital channels to their customers, and helping consumers send, receive and use

money more easily. Its website presents the company as a consulting firm in IT strategy and organisation that supports companies in their digital transformation, with the motto “Vision, passion and excellence in everything we do”.

1.1.3 Products and services

The products of IWOMI Technologies serve two groups of users, as shown in Figure 1: financial institutions on one side, consumers and merchants on the other.

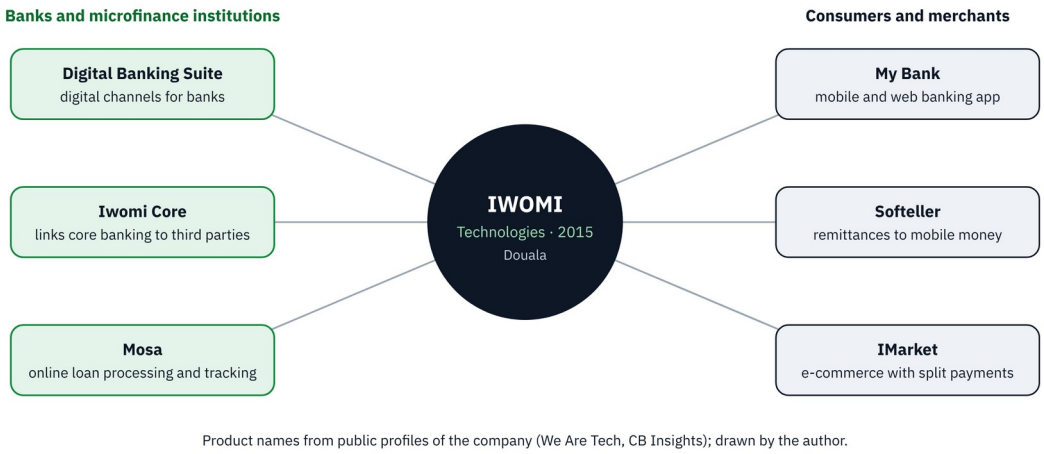


Figure 1: *Main products of IWOMI Technologies (source: author)*

Table 2: *Products of IWOMI Technologies*

Product	Users	Function
Digital Banking Suite	Banks, microfinance institutions	Digital channels (web and mobile) offered by an institution to its customers
Iwomi Core	Banks, microfinance institutions	Integration layer that connects a core banking system to third-party services (mobile money, payment platforms)
Mosa	Banks and their customers	Secure online processing and tracking of loan requests; customers manage their loans remotely
My Bank	Bank customers	Mobile and web banking application
Softeller	Consumers	Instant transfers from abroad to mobile money accounts, payment of bills (ENEO, Camwater)
IMarket	Merchants and consumers	E-commerce with split payments

Besides these products, the company offers services of IT consulting, integration and customisation of its solutions for each institution, training of the users and technical assistance in digital transformation projects.

1.1.4 Organisation

IWOMI Technologies is a small, technical company. Under the managing director, the work is organised around a few teams: software development (web, mobile and back-end integration with banking systems), project management and customer support for the institutions that use the products, and business development and administration. The interns are attached to the development team and

work under the supervision of a senior member of staff; during my internship my professional supervisor was Mr. KEN.

Table 3: Organisation of IWOMI Technologies as observed during the internship

Unit	Main tasks
General management	Strategy, partnerships with banks and investors, validation of projects
Software development	Design, development and maintenance of the products; integration with core banking systems; supervision of interns
Projects and customer support	Deployment of the solutions at each institution, training of users, technical assistance
Business development and administration	Prospecting, contracts, finance and human resources

This organisation is described from my observation during the internship; the exact organisation chart of the company was not provided in writing.

1.2 Activities carried out during the internship

The internship lasted nine weeks, from July 1 to August 31, 2026. Figure 2 shows how the time was distributed between the familiarisation with the company and its website, the study of the security tools, the development of the scanner and its validation.



Figure 2: Planning of the internship (nine weeks) (source: author)

Table 4: Activities carried out week by week

Week	Activities	Result
1 (Jul 1–3)	Reception, presentation of the team and of the products; installation of the work environment (Windows, Python, code editor); study of the company website	Work environment ready; list of website sections
2–3 (Jul 6–17)	Reproduction of the IWOMI website in HTML and CSS: header and navigation, home banner, “Who are we?” section, products, footer; adjustments of colours, typography and spacing	Static reproduction of the website served locally (Figures 3 and 4)

Week	Activities	Result
3–4 (Jul 13–24)	Documentary research on web security and the OWASP Top 10; installation and configuration of OWASP ZAP and Nikto	Tools installed; notes on their use
4–5 (Jul 20–31)	Manual assessment of an authorized website with the browser tools, OWASP ZAP and Nikto; ZAP report generated; analysis of the time spent	First ZAP report; list of the problems of the manual process
5 (Jul 27–31)	Requirements and design of the scanner: use cases, architecture, scan pipeline, data model	Requirements (Chapter 3) and diagrams (Chapter 4)
6–7 (Aug 3–14)	Back end in Python/Flask: target validation, authorization, crawler, header, cookie, TLS and redirect checks, URL risk heuristic	Increment 1: core checks working
7–8 (Aug 10–21)	Interface (templates, progress page), wrappers for ZAP, Nikto and Nmap, history, PDF report	Increment 2: complete workflow
8–9 (Aug 17–28)	Persistence of the scan status, score and OWASP categories, laboratory website, 21 automated tests, bug fixes, measurements	Increment 3: version 2.1, tests 21/21
9 (Aug 24–31)	Documentation, demonstration to the supervisor, writing of this report	README, report

1.2.1 First assignment: reproduction of the company website

During weeks 2 and 3, before the main project, I reproduced the home page of the IWOMI Technologies website in HTML5 and CSS3 to get familiar with the company's image and products. The page reproduces the header and navigation bar (Home, About us, Products, Services, Realizations, Careers, Blogs, Contact), the home banner, the "Who are we?" section and the footer. It is divided into semantic sections (`header`, `nav`, `section`, `footer`); the layout uses Flexbox and Grid, media queries adapt it to small screens, and the colours of the brand are defined once as CSS variables.

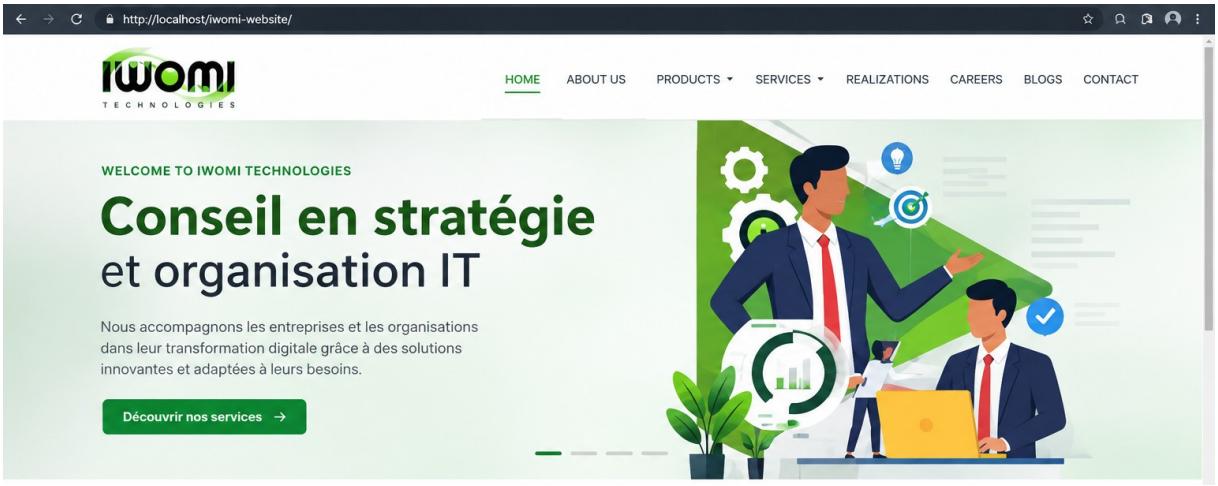


Figure 3: *Reproduction of the IWOMI website: navigation bar and banner (source: author)*

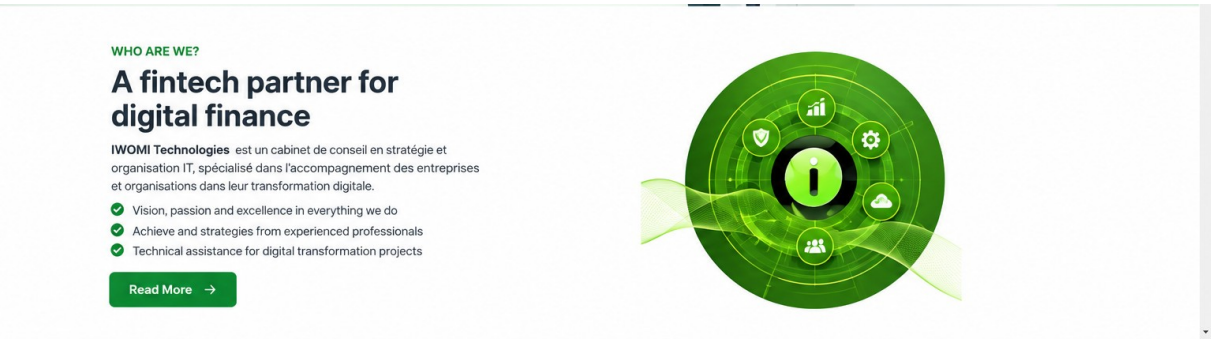


Figure 4: *Reproduction of the IWOMI website: "Who are we?" section (source: author)*

The main difficulty was reproducing the spacing and the font sizes so that the page "feels" like the original; I solved it by comparing both pages side by side in the browser at the same width. This first assignment also made me look at the company website as a user would – which naturally led to the question of its security, the topic of this report.

1.2.2 Main project: the web application vulnerability scanner

From week 3 onwards, the internship was devoted to web security: study of the tools, manual assessment of an authorized website, then design, development and validation of the scanner presented in Part Two.

1.3 Choice and justification of the topic

The topic was chosen from a need observed during the internship rather than imposed in advance.

The **vulnerability scanner** came from my own experience during weeks 4 and 5. Assessing one website by hand took me several hours, most of them spent configuring tools and comparing their outputs rather than understanding the risks. For a company whose clients are banks, being able to check a web interface quickly and in the same way every time – before a demonstration, after a deployment, or when a client asks – has a direct value. The topic also matched my training in computer engineering: it combines networking (HTTP, TLS, ports), programming (Python, Flask, SQL) and security. Finally, it could be carried out in nine weeks with free tools and without any risk for third parties, since every test could be done on an authorized laboratory target.

Conclusion

IWOMI Technologies is a Cameroonian fintech that builds digital banking solutions for financial institutions and consumers. During the nine weeks of the internship I first reproduced its website to get familiar with it, then studied the usual web security tools, and observed the limits of a manual security check, which led to the topic of this report. The next chapter reviews the concepts and tools on which the work relies.

CHAPTER 2
OVERVIEW OF THE TOPIC STUDIED (LITERATURE REVIEW)

Introduction

This chapter presents the notions needed to understand the work: the principles of web application security and the OWASP Top 10, the security controls checked by the scanner, the existing scanning tools, and the legal and ethical framework of security testing in Cameroon.

2.1 Web application security

2.1.1 Objectives of security

Security aims at three properties, often called the CIA triad: **confidentiality** (only authorized persons read the data), **integrity** (data are not modified without authorization) and **availability** (the service works when it is needed). A web application exposes these properties to anybody on the Internet: the browser, the network, the web server, the application and its database are all possible points of attack.

2.1.2 The OWASP Top 10 (2021)

The Open Worldwide Application Security Project (OWASP) is a non-profit foundation that publishes free references on application security. Its **Top 10** lists the ten most critical categories of web application security risks, based on data collected from many organisations. The 2021 edition is the reference used by the scanner to classify its findings.

Table 5: The OWASP Top 10 (2021) and what the scanner checks

Category	Short description	Covered by the scanner
A01 Broken Access Control	Users act outside their permissions	Redirect chains (partial)
A02 Cryptographic Failures	Sensitive data exposed by missing or weak cryptography	HTTPS, TLS version, certificate
A03 Injection	SQL, OS or script injection (XSS)	Only through OWASP ZAP (optional)
A04 Insecure Design	Missing security controls in the design	No (needs a manual review)
A05 Security Misconfiguration	Missing hardening, default settings, verbose headers	Security headers, cookies, Server header
A06 Vulnerable and Outdated Components	Known vulnerable versions of software	Through Nikto and Nmap (optional)
A07 Identification and Authentication Failures	Weak login and session management	Cookie flags (partial)
A08 Software and Data Integrity Failures	Untrusted updates, insecure deserialization	No
A09 Security Logging and Monitoring Failures	Attacks not detected	No

Category	Short description	Covered by the scanner
A10 Server-Side Request Forgery	The server fetches an attacker-chosen URL	Only through OWASP ZAP (optional)

2.2 Security controls checked by the scanner

2.2.1 HTTP security headers

Modern browsers include protections that a website switches on by sending HTTP response headers. When a header is missing, the protection is simply not used. Table 6 summarises the headers checked and the severity given by the scanner when they are missing.

Table 6: *Security headers checked by the scanner*

Header	Protects against	Example of value	Severity if missing
Content-Security-Policy	Cross-site scripting, injection of resources	default-src 'self'	Medium
X-Frame-Options	Clickjacking (page inside a hidden frame)	DENY	Low
X-Content-Type-Options	MIME-type sniffing	nosniff	Low
Referrer-Policy	Leak of URLs to other sites	strict-origin-when-cross-origin	Low
Permissions-Policy	Abuse of camera, microphone, location	camera=(), microphone=()	Low
Strict-Transport-Security	Downgrade from HTTPS to HTTP (HTTPS sites only)	max-age=31536000	Low

The scanner also reports a Content-Security-Policy containing `unsafe-inline` or `unsafe-eval` (Medium), which keeps the header but cancels most of its protection, and a `Server` header that reveals the product or version of the web server (Informational).

2.2.2 Cookies

Session cookies identify a logged-in user. The **Secure** attribute prevents the browser from sending them over unencrypted HTTP; the **HttpOnly** attribute prevents JavaScript from reading them, which limits the impact of a cross-site scripting flaw. The **SameSite** attribute limits their sending from other sites. A missing Secure or HttpOnly attribute is reported as a Low finding.

2.2.3 Transport security (HTTPS and TLS)

HTTPS protects the traffic between the browser and the server with the TLS protocol: it encrypts the data and, thanks to the certificate, proves the identity of the server. A site still served in plain HTTP exposes passwords and session cookies on the network (Medium). Among TLS versions, SSLv3, TLS 1.0 and TLS 1.1 are obsolete and must be disabled; TLS 1.2 and TLS 1.3 are the current versions. A failed handshake often reveals an expired or mismatching certificate.

2.2.4 Redirects

A redirect (HTTP status 301, 302, 307 or 308 with a `Location` header) sends the browser to another address. Long redirect chains slow down the site, make the path of a request hard to audit, and can hide a hop through HTTP or through another domain. The scanner follows up to eight hops and reports a chain of more than two redirects.

2.2.5 Phishing and suspicious URLs

Phishing sites imitate a trusted brand – very often a bank or a mobile money service – to steal credentials. Their URLs show typical signs that can be checked without even opening the page: an IP address instead of a domain name, an @ sign that hides the real destination, many subdomains or hyphens, a very long URL, percent-encoded characters, a punycode (IDN) domain that imitates another one, a top-level domain often used for abuse, or words such as “login”, “verify” or “secure” in the host name. A score built on such signs is a **heuristic**: it helps to prioritise, but it can neither prove that a site is fake nor that it is genuine.

2.3 Existing tools for web security assessment

Many tools exist, each with its own scope. Table 7 compares those studied during the internship.

Table 7: Comparison of existing web security tools

Tool	Type	Strengths	Limits for our need
OWASP ZAP	Open-source DAST proxy and scanner	Spider, passive and active scan, many rules, HTML/JSON reports	Heavy (Java), long configuration; active scan sends attacks and needs strict authorization
Nikto	Open-source web server scanner (Perl)	Thousands of checks for dangerous files and outdated servers	Text output only, many false positives, noisy
Nmap	Open-source network scanner	Open ports, service and version detection	Not a web application scanner; results unrelated to the pages
Burp Suite Community	Proxy for manual testing	Excellent for an expert tester	Manual work; automatic scanner only in the paid edition
Commercial scanners (e.g. Acunetix, Nessus)	Commercial DAST / vulnerability scanners	Wide coverage, dashboards, support	Licence of several thousand dollars per year; too expensive for a small team
Online header checkers	Web services	Fast check of headers	One page only, no history, the target is sent to a third party

None of these tools, used alone, answers the need described in the problem statement: a single place where an authorized check of a site is started, followed, stored, scored and exported, while the specialised tools remain usable when they are installed.

2.4 Technologies of the solution

- **Python 3** – a readable language with a rich standard library (`ssl`, `socket`, `html.parser`, `sqlite3`, `ipaddress`) for network and text processing.
- **Flask** – a lightweight Python web framework: routes, Jinja2 templates, form handling. It is enough for a tool used by a small team and easy to read for an intern.
- **Requests** – the reference Python library for HTTP requests (sessions, redirects, cookies, timeouts).
- **SQLite** – an embedded database stored in one file, with no server to install; well suited to a single-user tool.
- **ReportLab** – a Python library that generates PDF documents.
- **pytest** – the Python test framework used for the 21 automated tests.

2.5 Development process

The classic **waterfall** model fixes all requirements at the start; it fits poorly a project where the problem is discovered while learning the tools. **Agile** methods such as Scrum need a team and a regular rhythm of reviews. I therefore chose an **iterative and incremental** model: after the requirements, the scanner was built in three increments, each one adding features on top of a tested base and each one shown to the supervisor. The **MoSCoW** method (Must, Should, Could, Won't) was used to order the requirements so that the essential ones were delivered first.

2.6 Legal and ethical framework

In Cameroon, **Law No. 2010/012 of 21 December 2010 on cybersecurity and cybercriminality** punishes, among other offences, unauthorized access to all or part of an information system, staying in it fraudulently, and hindering or disturbing its functioning. Sending attack requests to a website without the authorization of its owner can therefore be an offence, even "to test". Personal data met during a test are also protected by **Law No. 2024/017 of 23 December 2024 on the protection of personal data**.

Professional practice adds clear rules: obtain a written authorization that defines the **scope** (which hosts, which period, which kinds of tests), prefer passive checks, never use the findings outside the mission, and report them only to the owner. These rules are built into the scanner: a scan cannot start without an explicit authorization checkbox, the crawler never leaves the host of the target, and the active tools only run when an operator has installed and configured them.

Conclusion

A large part of web risks comes from configuration mistakes that can be detected by reading normal responses: headers, cookies, TLS and redirects. Existing tools cover these checks in a scattered way, with different formats and no common score. Within the legal framework of Cameroon, an

authorized, mostly passive tool that gathers these checks is both useful and responsible. The next part analyses the existing practice and presents the solution.

PART TWO:
EMPIRICAL STUDY

CHAPTER 3
DIAGNOSTIC OF THE EXISTING SYSTEM AND REQUIREMENTS SPECIFICATION

Introduction

This chapter analyses the existing practice of website security assessment met during the internship, identifies its problems, and specifies the requirements of the solution.

3.1 The existing security assessment process

3.1.1 Description

Figure 5 shows how a website was checked before the project. The analyst first read the headers and cookies in the browser developer tools, then ran OWASP ZAP (spider and scan through its graphical interface), Nikto and Nmap in a terminal, and finally copied the important points into a Word or Excel document for the development team.

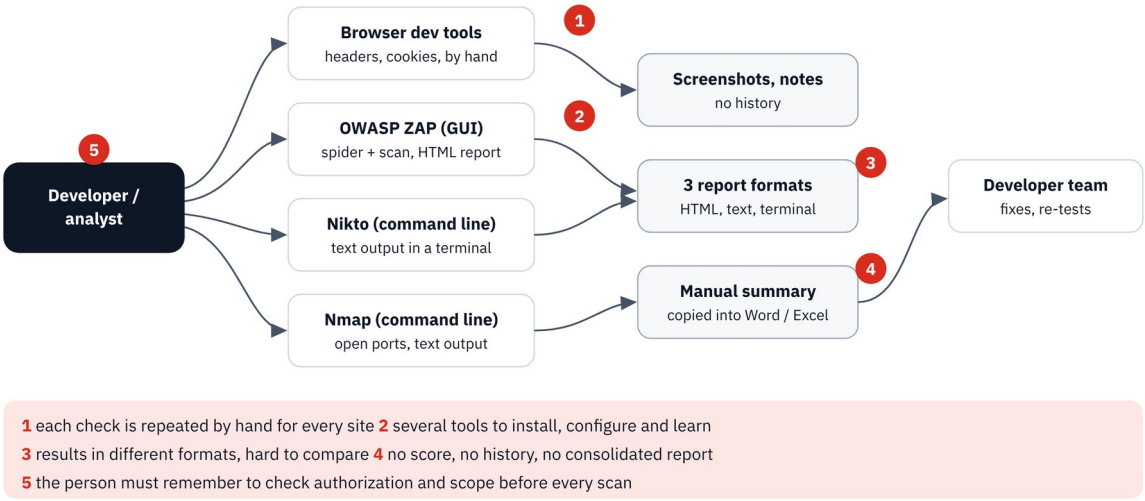


Figure 5: *Existing manual assessment process and its problems (source: author)*

3.1.2 Observation of a manual assessment

During weeks 4 and 5, I carried out this process completely on an authorized website indicated by my supervisor. Table 8 summarises my observations.

Table 8: *Observation of one manual assessment*

Step	Time spent (approx.)	Observation
Installation and configuration of ZAP, Nikto and Nmap (first time)	1 to 2 days	Java for ZAP, Perl for Nikto; paths and options to learn
Reading headers and cookies in the browser	20 to 30 min per site	Page by page; easy to forget one header

Step	Time spent (approx.)	Observation
ZAP spider and scan	30 min to more than 1 h	Report in HTML; many alerts to sort
Nikto and Nmap	15 to 30 min	Raw text in a terminal
Writing the summary	1 h or more	Copy and paste, no common severity scale, no score

3.1.3 Problems identified

- **P1 – Repetitive manual work**: the same checks are repeated by hand for every site and every new version.
- **P2 – Several tools to install and learn**: each tool has its own language, options and interface.
- **P3 – Heterogeneous results**: HTML, plain text and screenshots, with different severity scales; findings appear twice.
- **P4 – No score and no history**: it is impossible to say whether a site improved since the last check.
- **P5 – Authorization depends on memory**: nothing in the process reminds the analyst to check that the target and the scope are authorized.

3.1.4 Strengths to keep

The manual process also has strengths: the specialised tools are powerful and free, and the analyst's judgement is irreplaceable to confirm a finding. The solution must therefore not replace these tools but organise them, and must present its results as indications to be confirmed.

3.2 Requirements specification

3.2.1 Actors

The main actor is the **authorized tester** (developer or analyst of the company). External systems are the **target website** and, optionally, **OWASP ZAP**, **Nikto** and **Nmap**.

3.2.2 Functional requirements

Table 9: *Functional requirements*

ID	Requirement	Priority
FR-01	The tester enters a target URL; only HTTP and HTTPS URLs with a valid host are accepted, and https:// is added when the scheme is missing.	Must
FR-02	A scan cannot start unless the tester ticks "I am authorized to test this target".	Must
FR-03	The scanner crawls the pages of the same host (40 pages at most) and lists the URLs and forms found.	Must
FR-04	The scanner checks the six security headers, a weak CSP and the disclosure of the Server header.	Must
FR-05	The scanner checks the Secure and HttpOnly attributes of cookies.	Must
FR-06	The scanner checks HTTPS, the negotiated TLS version and the handshake.	Must

ID	Requirement	Priority
FR-07	The scanner follows the redirects (8 hops at most) and reports long chains.	Should
FR-08	The scanner evaluates the risk of the URL with phishing heuristics.	Should
FR-09	A separate page evaluates the risk of any URL without sending any request to it.	Should
FR-10	The scanner runs OWASP ZAP, Nikto and Nmap when they are configured, and says clearly when a stage was skipped.	Should
FR-11	The scan runs in the background; a progress page shows the stage, the percentage and the counters.	Must
FR-12	Findings are deduplicated, sorted by severity and classified by OWASP Top 10 category.	Must
FR-13	Each scan receives a score from 0 to 100 and a grade from A to F.	Should
FR-14	Each finding shows a description, the evidence and a recommendation.	Must
FR-15	Scans, pages and findings are stored and listed in a history.	Must
FR-16	The results of a scan can be downloaded as a PDF report.	Should
FR-17	Authenticated scans (with a login) and scheduled scans.	Won’t (this version)

3.2.3 Non-functional requirements

Table 10: *Non-functional requirements*

ID	Requirement	Measure
NFR-01 Safety	Only authorized targets; the crawler stays on the host of the target; built-in checks are passive	Tests FR-02 and same-host crawl
NFR-02 Performance	The built-in passive checks of a small site end in less than 5 s on a local network	Median measured on the lab site
NFR-03 Robustness	An unreachable target or a failing tool never stops the scan; the error is shown as a finding	Error paths tested
NFR-04 Persistence	Results survive a restart of the server	Test after reload from the database
NFR-05 Honesty of results	No result claims that a site is “safe” or “real”; skipped stages are reported	Labels tested
NFR-06 Usability	A scan is started in fewer than 3 actions; results are grouped and readable	Observation
NFR-07 Portability	Runs on Windows and Linux with Python 3.10+ and 3 dependencies	Installation on both
NFR-08 Maintainability	One module per check; automated tests	Code structure, 21 tests

3.3 Feasibility and risks

Technical feasibility: all the checks rely on the Python standard library and Requests; the external tools are free. **Economic feasibility**: no licence is needed and the tool runs on the analyst’s computer. **Time**: the MoSCoW priorities fit the “Must” requirements into the six weeks of development.

Table 11: Main risks of the project and their mitigation

Risk	Impact	Mitigation
Scanning a site without authorization	Legal and reputational	Mandatory checkbox; tests only on the lab site and on the authorized site
Active tools disturbing a production site	Service interruption	External tools optional and off by default; passive built-in checks
False positives presented as facts	Wrong decisions	Evidence and recommendation for each finding; heuristic labels never say “real”
Losing results when the server stops	Work lost	Status and results stored in SQLite (version 2.1)
Not enough time	Incomplete tool	Iterative increments, MoSCoW priorities

Conclusion

The diagnosis showed five problems of the manual assessment: repetition, many tools, heterogeneous results, no score or history, and an authorization left to memory. Seventeen functional and eight non-functional requirements answer them. The next chapter presents the solution built to meet them.

CHAPTER 4
PROPOSED SOLUTION AND IMPLEMENTATION

Introduction

This chapter presents the Web Application Vulnerability Scanner: its design, the implementation, the tests, the installation procedure, the cost and the limits of the solution.

4.1 Presentation of the scanner

The **Web Application Vulnerability Scanner** is a web application used in the browser. The tester enters a URL, confirms the authorization and starts the scan; a progress page follows the stages live; the results page shows the score, the counters, the findings grouped by OWASP category with their evidence and recommendation, the pages crawled and the state of the external tools; the history keeps every scan and each scan can be exported as a PDF report. A separate page evaluates the risk of a URL from its text only.

4.1.1 Evolution of the project

Table 12: *Versions of the scanner*

Version	Period	Content
1.0	Weeks 5–6	Flask form, target restricted to the laboratory (localhost/private addresses), authorization checkbox, first header checks
2.0	Weeks 6–8	Background scan with progress page, same-host crawler, cookies, TLS, redirects, URL heuristics, ZAP JSON parsing, Nikto and Nmap stages, history, PDF
2.1	Weeks 8–9	Status stored in SQLite, score and grade, OWASP categories, URL check page, new interface, laboratory site, 21 tests, bug fixes

4.2 Design

4.2.1 Use case diagram

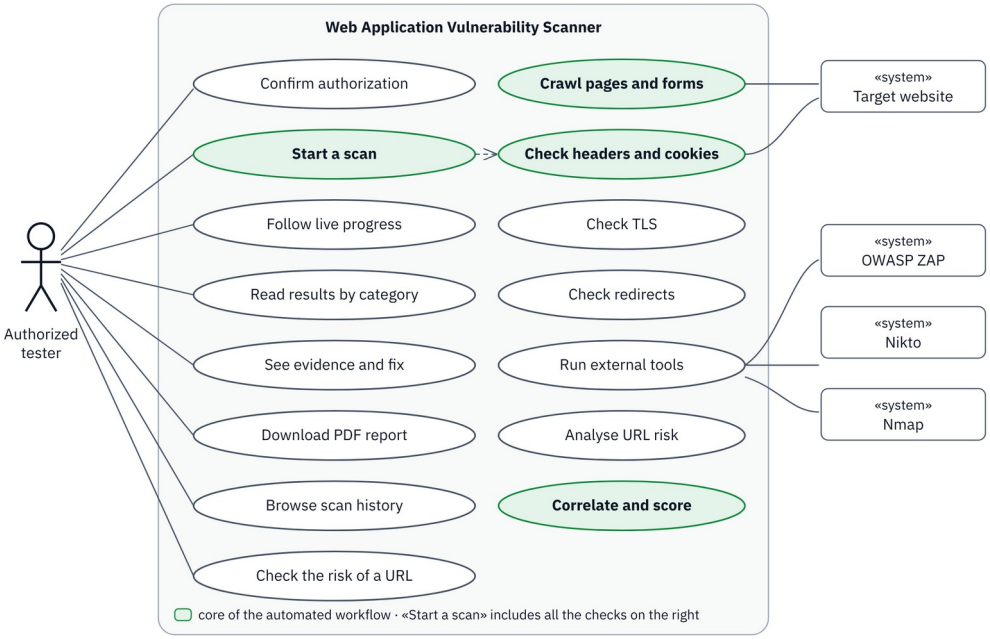


Figure 6: *Use case diagram of the scanner (source: author)*

4.2.2 Architecture

The application follows a simple layered architecture (Figure 7): the browser displays the Jinja2 pages; Flask (`app.py`) receives the requests and starts each scan in a background thread; the `scanner/` package contains one module per check; `analyzer.py` normalises, categorises and scores the findings; `database/db.py` stores them in SQLite and `reports/report.py` produces the PDF.

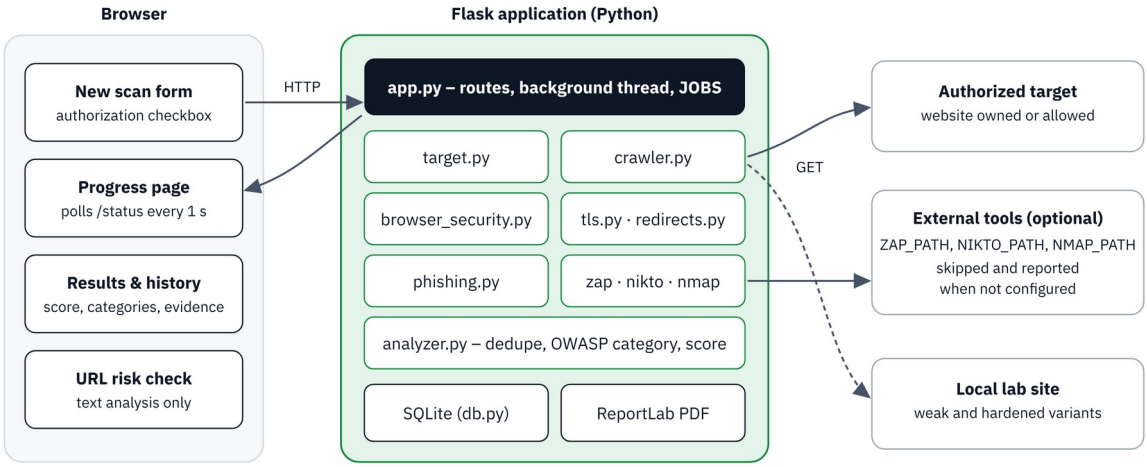


Figure 7: *Architecture of the scanner (source: author)*

4.2.3 Scan pipeline

A scan goes through the stages of Figure 8. Each stage writes its name, its percentage and the counters to the database, which the progress page reads every second. If a stage fails, the findings already collected are kept and the error is shown.

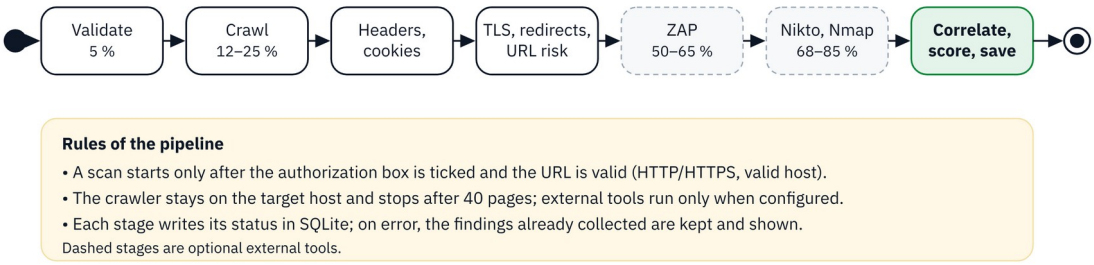


Figure 8: *Stages of a scan (source: author)*

4.2.4 Sequence diagram

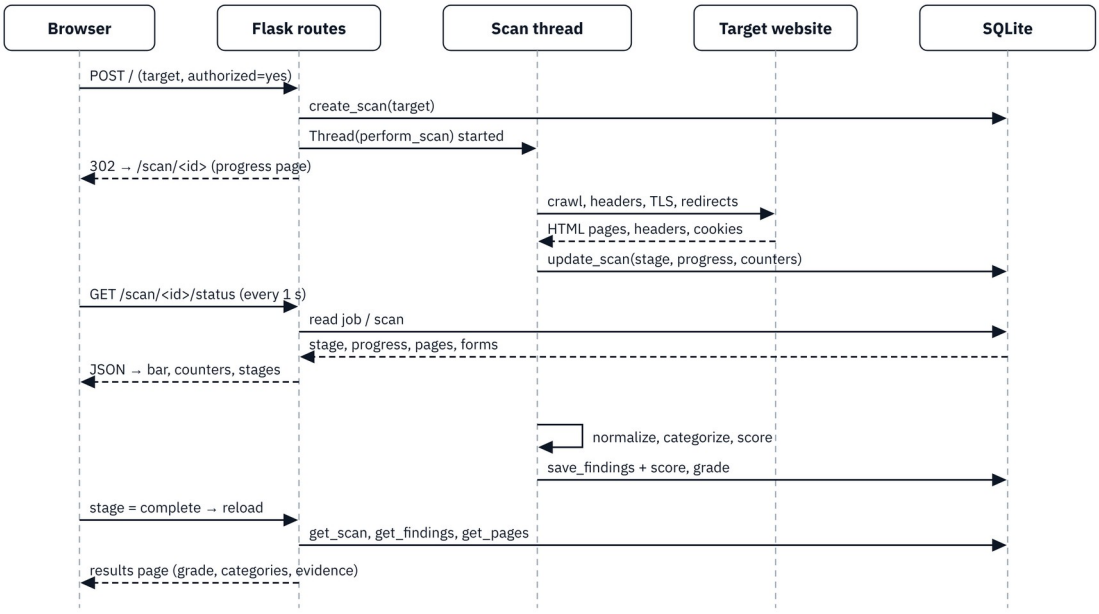


Figure 9: *Sequence diagram "start a scan and follow its progress" (source: author)*

4.2.5 Data model

Three tables are enough (Figure 10): `scans` for each scan and its live status, `findings` for its findings and `pages` for the pages crawled. When the application starts on an older database, the missing columns of version 2.1 are added automatically, so that no history is lost.

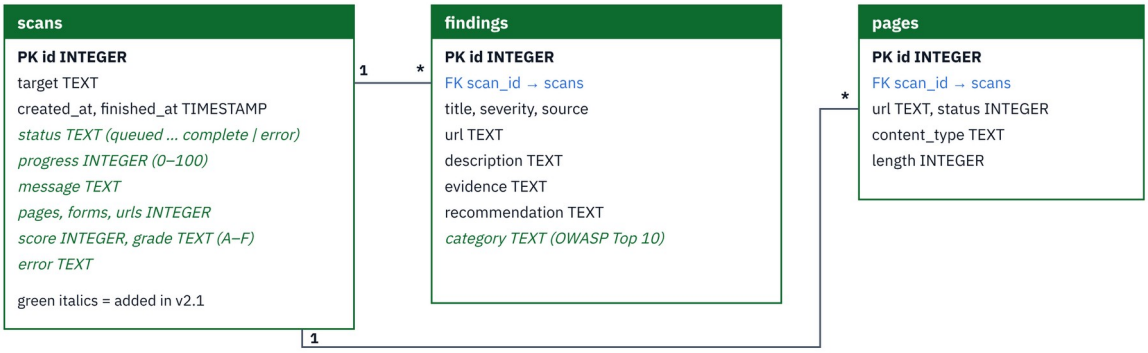


Figure 10: Data model (SQLite) (source: author)

4.2.6 Code organisation

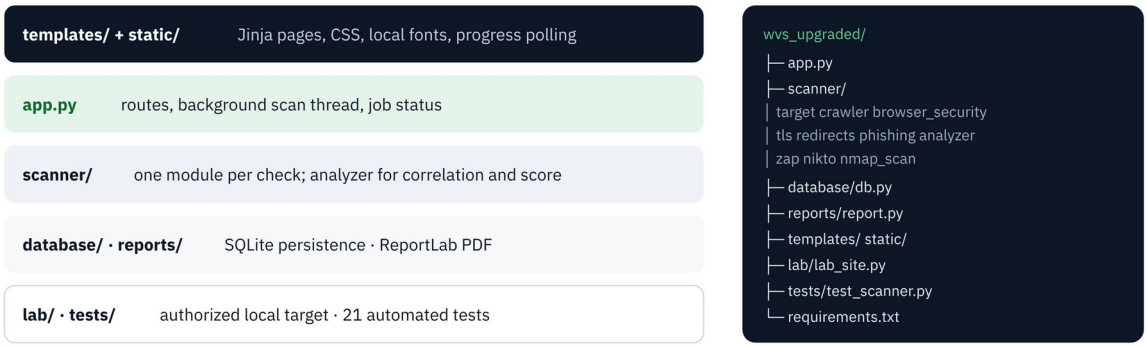


Figure 11: Layers and folders of the project (source: author)

4.3 Implementation

4.3.1 Target validation and authorization

`scanner/target.py` accepts only HTTP and HTTPS URLs with a valid host name and adds `https://` when no scheme is given. In `app.py`, the POST route refuses to start a scan when the authorization box is not ticked. This double barrier answers problem P5.

4.3.2 Crawler

The crawler (`scanner/crawler.py`) is a breadth-first search limited to the host of the target and to 40 pages. It parses each HTML page with `html.parser` to collect links, resources and forms (action and method), ignores fragments, and records for each page its status, content type and size.

4.3.3 Passive checks

`browser_security.py` reads the headers and cookies of the response and creates one finding per missing protection; HSTS is only required on HTTPS. `tls.py` opens a TLS connection with the default secure context of Python and reads the negotiated protocol and cipher. `redirects.py` follows the `Location` headers one by one. `phishing.py` computes the URL risk score:

```
def analyze_url(target):
    p = urlparse(target); host = (p.hostname or '').lower(); reasons = []; score = 0
    try: ipaddress.ip_address(host); reasons.append('Host is an IP address ...'); score += 25
    except ValueError: pass
    if len(target) > 100:  score += 10; reasons.append('URL is unusually long.')
    if '@' in target:      score += 25; reasons.append('URL contains @ ...')
    if host.startswith('xn--') or '.xn--' in host: score += 20  # punycode / IDN
    ...
    if score >= 60:   label = 'HIGH RISK — LIKELY FAKE'
    elif score >= 30: label = 'SUSPICIOUS / UNVERIFIED'
    else:             label = 'LOW RISK (HEURISTIC CHECK)'   # never "real website"
```

4.3.4 External tools

`zap.py`, `nikto.py` and `nmap_scan.py` run the tools with `subprocess` only when the variables `ZAP_PATH`, `NIKTO_PATH` and `NMAP_PATH` are defined, with a time limit (15 min for ZAP). The JSON report of ZAP is parsed alert by alert instead of being discarded. When ZAP is not configured, the scanner adds an informational finding “OWASP ZAP not configured” instead of silently pretending that the stage ran.

4.3.5 Correlation, categories and score

`analyzer.py` normalises the severities, removes duplicates (same title, URL and severity), sorts the findings and gives each one its OWASP Top 10 category. The score is deliberately simple so that it can be explained to a client:

Table 13: Computation of the security score

Severity	Penalty per finding	Maximum penalty
Critical	25	100
High	15	60
Medium	8	32
Low	3	12
Informational	0	0
Score = max(0, 100 − sum of penalties)	**Grade**: A ≥ 90, B ≥ 75, C ≥ 60, D ≥ 40, F < 40	

The cap per severity prevents twenty minor findings of the same kind from hiding the real picture: a site with many Low findings but no serious one still gets a fair grade.

4.3.6 Background execution and persistence

A scan can last from less than a second to several minutes when ZAP is enabled. The POST route therefore creates the scan in the database, starts `perform_scan` in a thread and immediately redirects to the progress page. In version 2.0 the status was kept only in a Python dictionary in memory, so a restart of the server lost every scan in progress or finished; in version 2.1 every update is written to the `scans` table.

4.3.7 Defects found and corrected

Table 14: Defects found during the internship and their correction

Defect	Cause	Correction
Python not recognised in the Windows terminal	Python not on the PATH; virtual environment not activated	Reinstallation with “Add to PATH”, `venv\Scripts\activate`
IndentationError in app.py	Mix of tabs and spaces	Re-indentation with 4 spaces, editor configured
History page crashed	Link built with `url_for('scan')`, a route that did not exist	Link to `scan_progress`; test added
`ftp://`… accepted as target	Scheme added only when missing, not checked	Refusal of any scheme other than http/https; 5 invalid inputs tested
Results lost after a restart	Status kept in memory only	Columns of status in SQLite
URL heuristic said “REAL WEBSITE”	Wording of the low-risk label	“LOW RISK (HEURISTIC CHECK)”; test checks the word “real” never appears
Start and end times inconsistent	`created_at` in UTC, `finished_at` in local time	Both stored in local time

4.4 Presentation of the interfaces

The interface uses a sober dark navy and green palette, with colours reserved for the severities. Its fonts are served locally, so the tool works without Internet access.

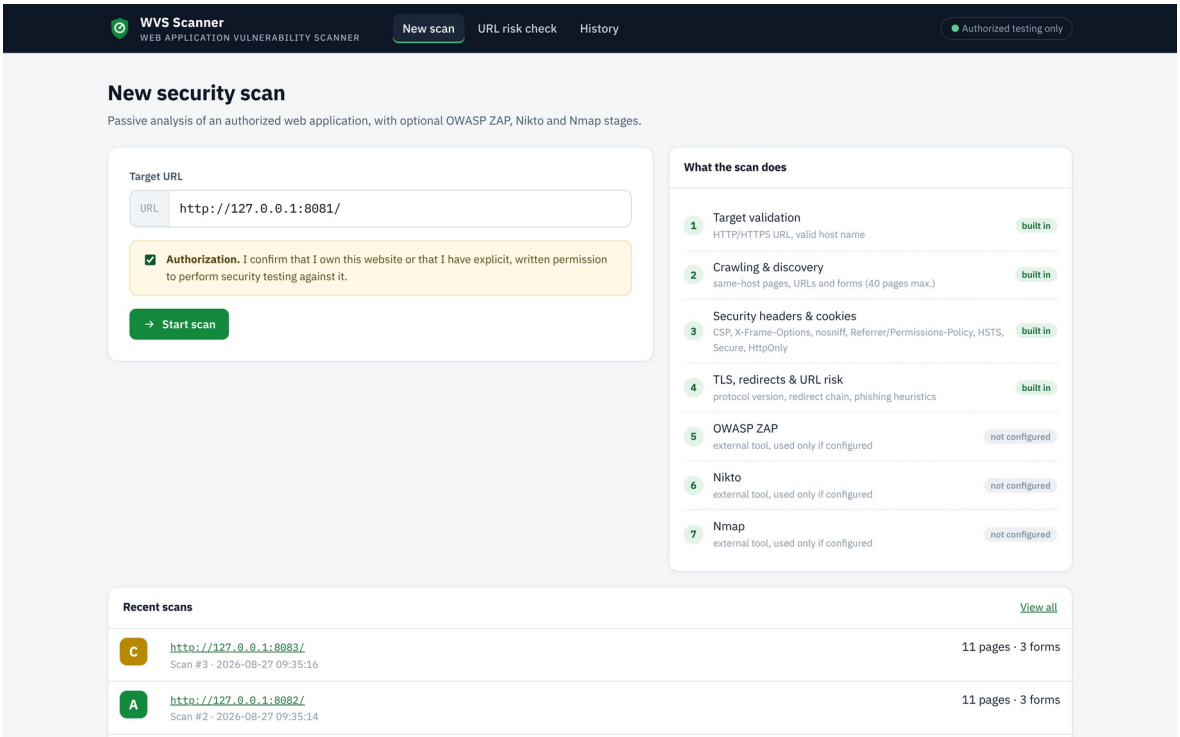


Figure 12: New scan page with the mandatory authorization and the recent scans (source: author)

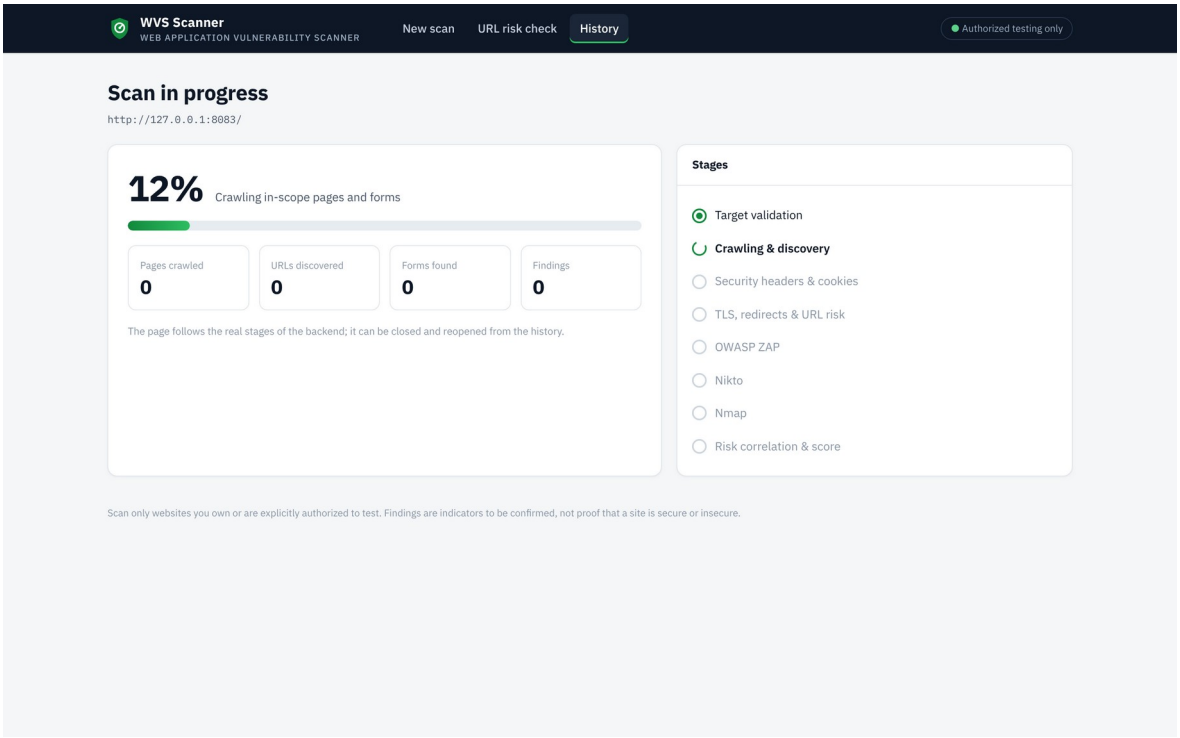


Figure 13: Progress page during a scan (source: author)

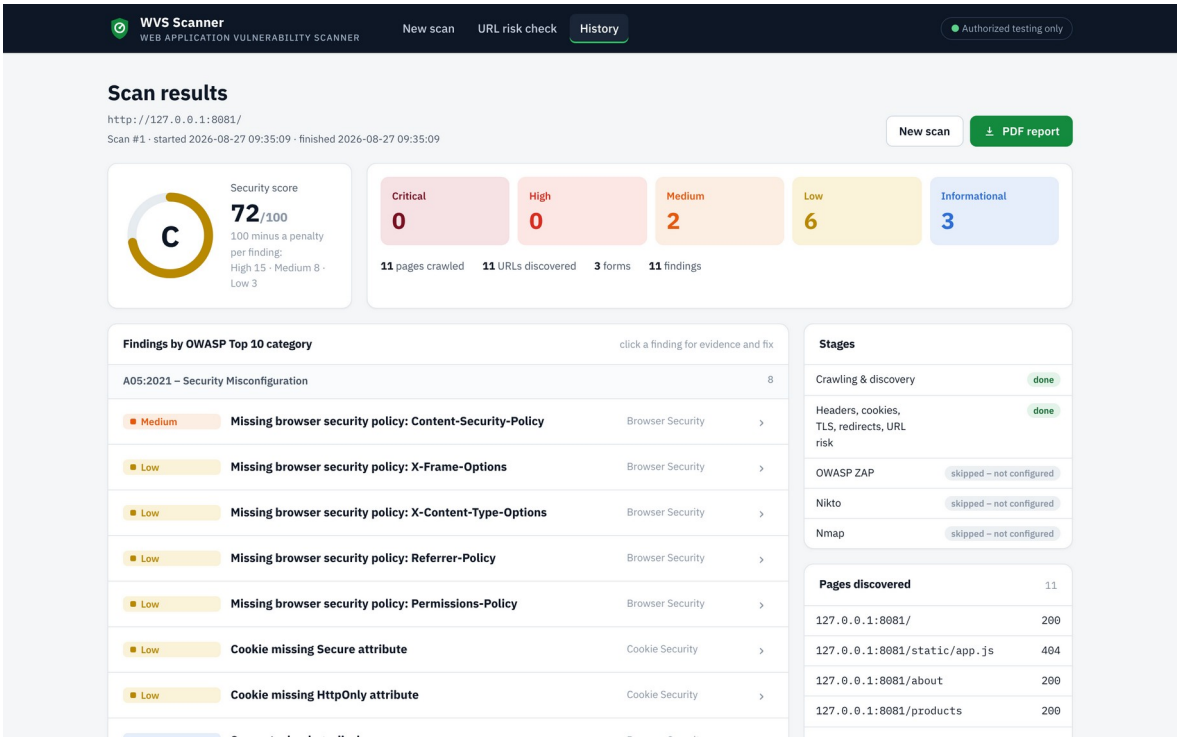


Figure 14: Results of the weak laboratory site: grade C, 72/100 (source: author)

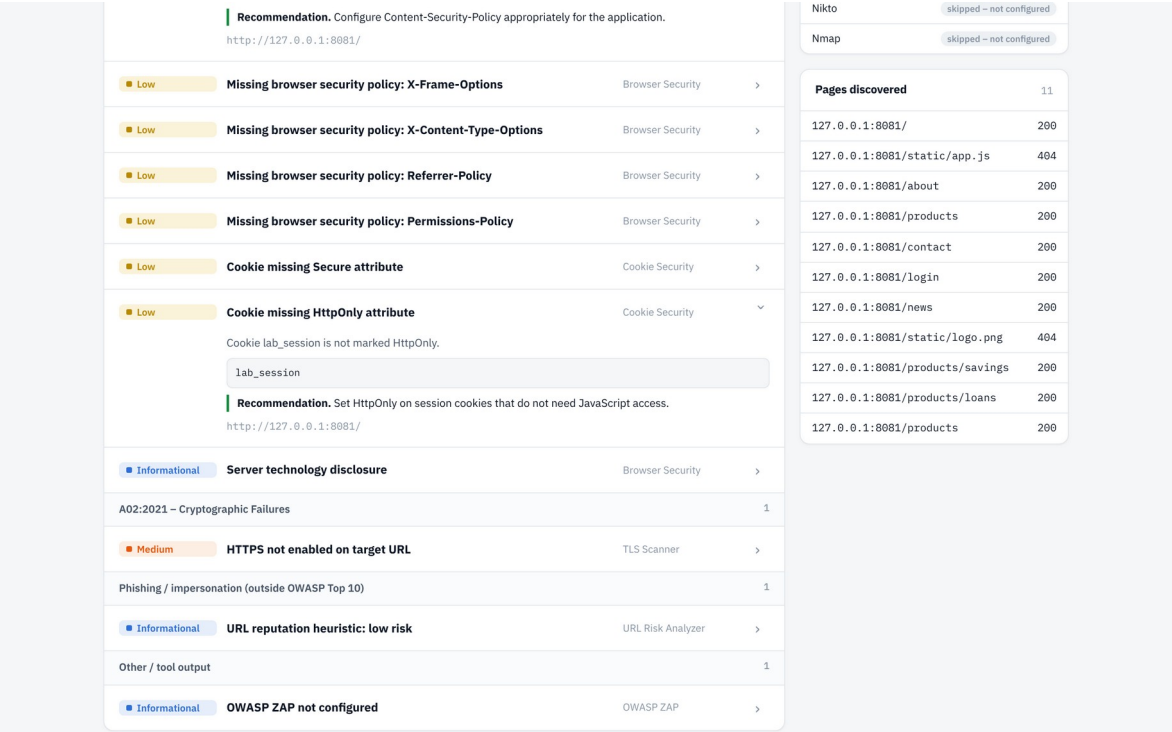


Figure 15: Detail of a finding with its evidence and recommendation (source: author)

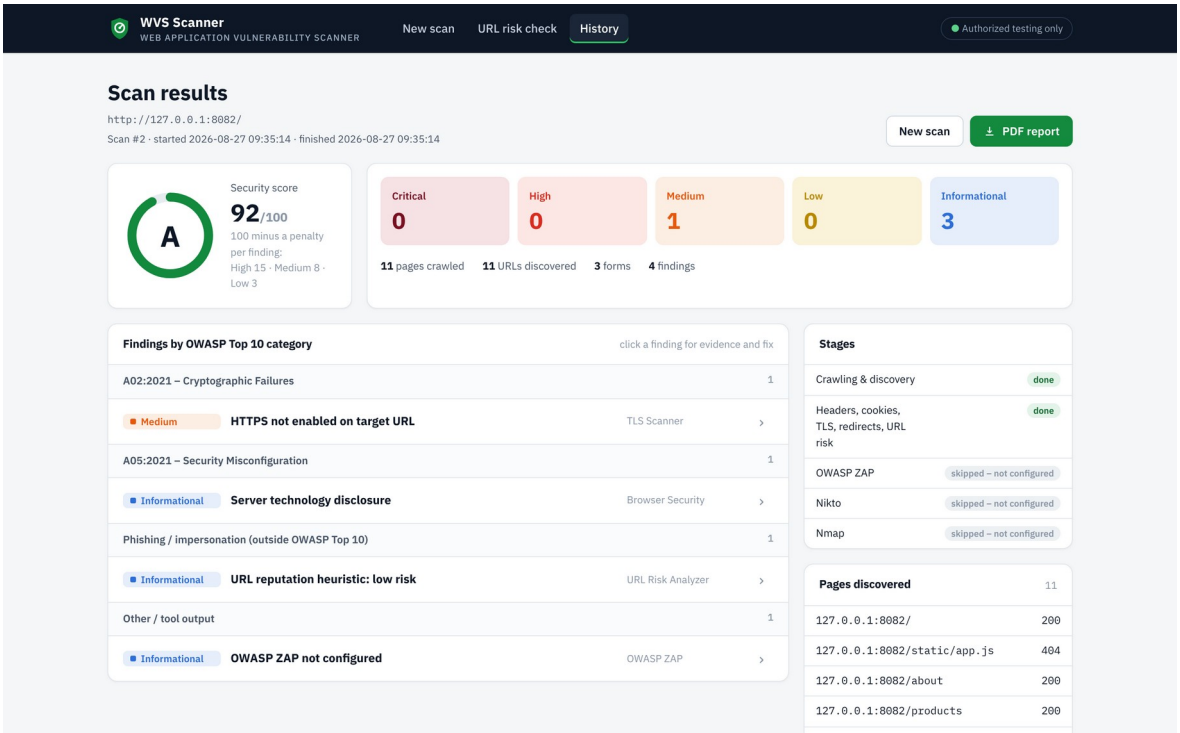


Figure 16: Results of the hardened laboratory site: grade A, 92/100 (source: author)

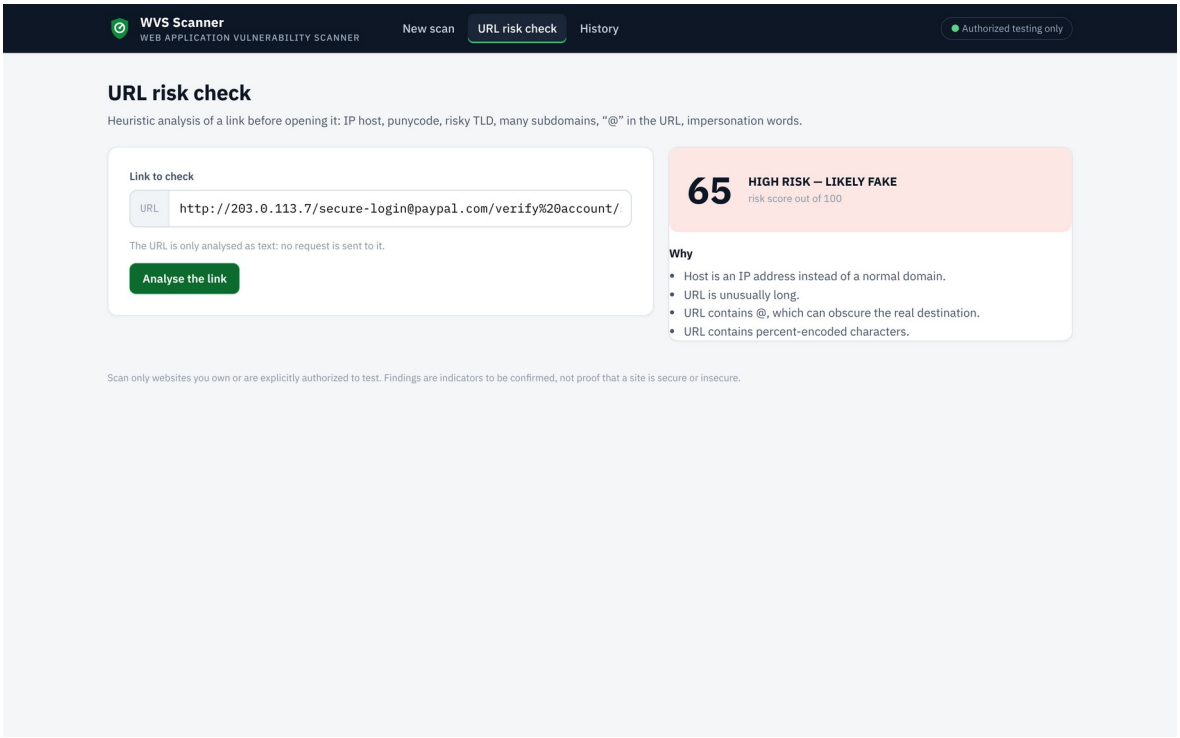


Figure 17: *URL risk check of a suspicious link: high risk (source: author)*

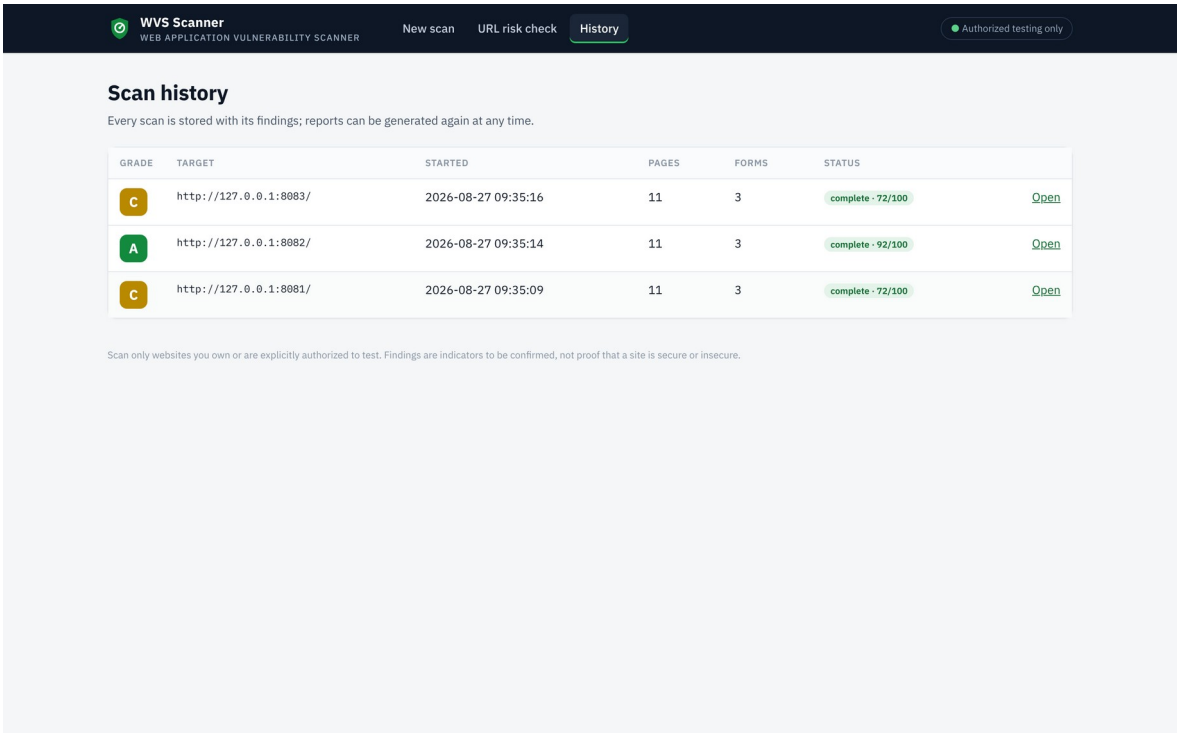
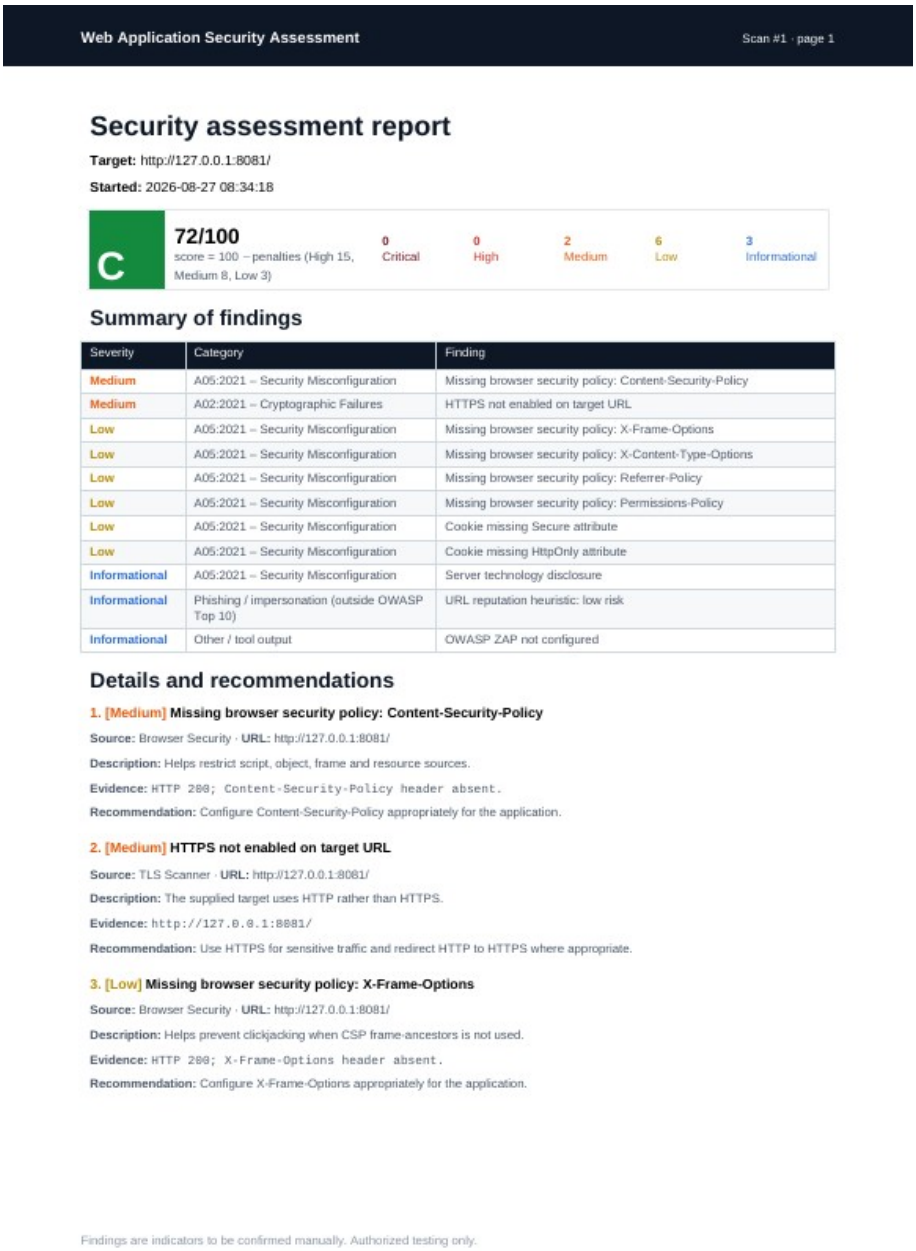


Figure 18: *History of the scans (source: author)*



Web Application Security Assessment — Scan #1 · page 1

Security assessment report

Target: http://127.0.0.1:8081/

Started: 2026-08-27 08:34:18

C — **72/100** score = 100 − penalties (High 15, Medium 8, Low 3)

0 Critical	0 High	2 Medium	6 Low	3 Informational

Summary of findings

Severity	Category	Finding
Medium	A05:2021 – Security Misconfiguration	Missing browser security policy: Content-Security-Policy
Medium	A02:2021 – Cryptographic Failures	HTTPS not enabled on target URL
Low	A05:2021 – Security Misconfiguration	Missing browser security policy: X-Frame-Options
Low	A05:2021 – Security Misconfiguration	Missing browser security policy: X-Content-Type-Options
Low	A05:2021 – Security Misconfiguration	Missing browser security policy: Referrer-Policy
Low	A05:2021 – Security Misconfiguration	Missing browser security policy: Permissions-Policy
Low	A05:2021 – Security Misconfiguration	Cookie missing Secure attribute
Low	A05:2021 – Security Misconfiguration	Cookie missing HttpOnly attribute
Informational	A05:2021 – Security Misconfiguration	Server technology disclosure
Informational	Phishing / impersonation (outside OWASP Top 10)	URL reputation heuristic: low risk
Informational	Other / tool output	OWASP ZAP not configured

Details and recommendations

1. [Medium] **Missing browser security policy: Content-Security-Policy**

Source: Browser Security · **URL:** http://127.0.0.1:8081/

Description: Helps restrict script, object, frame and resource sources.

Evidence: `HTTP 200; Content-Security-Policy header absent.`

Recommendation: Configure Content-Security-Policy appropriately for the application.

2. [Medium] **HTTPS not enabled on target URL**

Source: TLS Scanner · **URL:** http://127.0.0.1:8081/

Description: The supplied target uses HTTP rather than HTTPS.

Evidence: `http://127.0.0.1:8081/`

Recommendation: Use HTTPS for sensitive traffic and redirect HTTP to HTTPS where appropriate.

3. [Low] **Missing browser security policy: X-Frame-Options**

Source: Browser Security · **URL:** http://127.0.0.1:8081/

Description: Helps prevent clickjacking when CSP frame-ancestors is not used.

Evidence: `HTTP 200; X-Frame-Options header absent.`

Recommendation: Configure X-Frame-Options appropriately for the application.

Findings are indicators to be confirmed manually. Authorized testing only.

Figure 19: *First page of the PDF report generated for the weak laboratory site (source: author)*

4.5 Tests and validation

4.5.1 Laboratory website

To test the scanner without touching any third-party system, I wrote a small Flask site (`lab/lab_site.py`) that imitates the website of a fictitious microfinance institution: eight pages (home, about, products, savings, loans, contact, login, news), three forms and a redirect chain. It is served on the local machine in two variants: **weak** (no security header, cookie without Secure and HttpOnly, four-hop redirect chain) and **hardened** (the same pages with the recommended headers and cookie flags and a single redirect). The expected findings are therefore known in advance.

4.5.2 Automated tests

The 21 tests (pytest) start both variants of the laboratory site and check each module and the complete workflow through the Flask test client. All 21 tests pass (Figure 20).

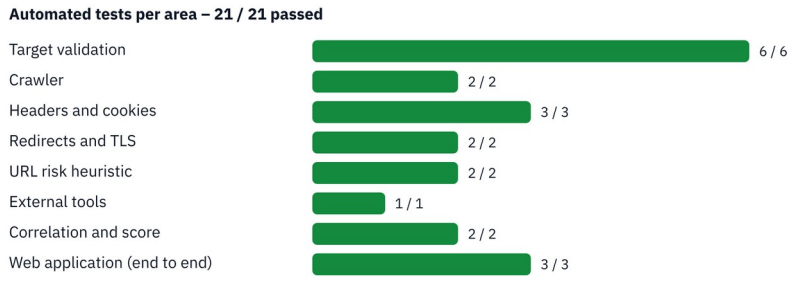


Figure 20: *Automated tests per area (21/21 passed) (source: author)*

4.5.3 Results on the laboratory site

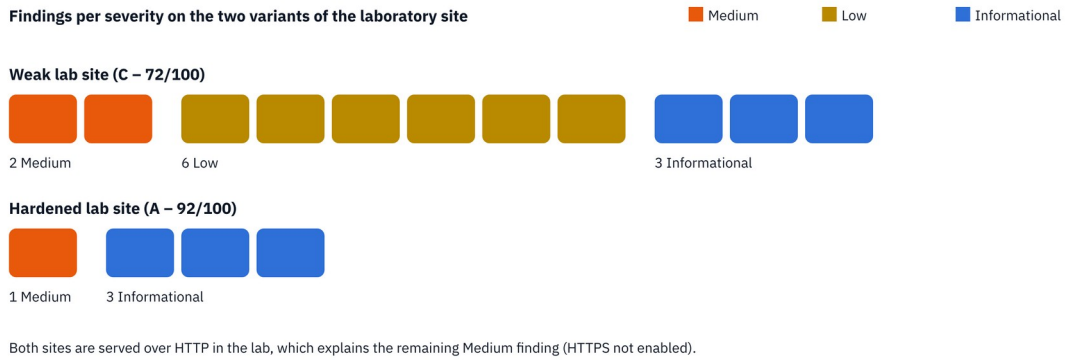


Figure 21: *Findings per severity on the two variants of the laboratory site (source: author)*

Table 15: *Results of the scanner on the two variants of the laboratory site*

Indicator	Weak variant	Hardened variant
Pages crawled / forms found	11 / 3	11 / 3
Medium findings	2 (HTTPS not enabled, CSP missing)	1 (HTTPS not enabled)
Low findings	6 (4 headers, 2 cookie flags)	0
Informational	3 (Server header, ZAP not configured, URL risk low)	3
Long redirect chain	Not counted on the home page; detected by the test on /old-offers (4 hops)	1 hop – not reported
Score / grade	**72 / C**	**92 / A**

The scanner finds every weakness placed in the weak variant and none that is absent from the hardened one. The only remaining Medium finding on the hardened variant is expected: the laboratory is served over HTTP on the local machine.

4.5.4 Performance

Table 16: Measured performance (median of repeated runs, local machine)

Operation	Result	Requirement
Built-in passive checks, weak site	51.4 ms	NFR-02: < 5 s ✔
Built-in passive checks, hardened site	35.6 ms	NFR-02 ✔
Crawl of the 11 pages	42.2 ms	–
URL risk check (text only)	≈ 6 µs	–
Complete scan with slowed site (0.6 s per response)	≈ 8 s, progress visible	NFR-06 ✔

These times are those of a local network; on a remote website they are dominated by the network latency of each request. When OWASP ZAP is enabled, its own scan lasts from several minutes to the 15-minute limit.

4.5.5 Coverage of the requirements

All the “Must” requirements (FR-01 to FR-06, FR-11, FR-12, FR-14, FR-15) and all the “Should” requirements are implemented and covered by tests, except FR-10 whose ZAP, Nikto and Nmap stages were tested only in their “not configured” path in the final version (see 4.8). FR-17 was deliberately left out of this version.

4.6 Installation and configuration

```
:: Windows (Command Prompt), in the project folder
python -m venv venv
venv\Scripts\activate
python -m pip install -r requirements.txt      :: Flask, requests, reportlab

:: optional external tools (only for authorized targets)
set ZAP_PATH=C:\Program Files\ZAP\Zed Attack Proxy\zap.bat
set NIKTO_PATH=C:\tools\nikto\program\nikto.pl
set NMAP_PATH=C:\Program Files (x86)\Nmap\nmap.exe

python app.py                                  :: then open http://127.0.0.1:5000/

:: laboratory site and tests
python lab\lab_site.py both                    :: weak on 8081, hardened on 8082
python -m pytest -q tests
```

4.7 Cost estimate of the solution

Table 17: Cost estimate of the solution (indicative)

Item	Quantity	Unit cost (FCFA)	Total (FCFA)
Software (Python, Flask, SQLite, ReportLab, OWASP ZAP, Nikto, Nmap)	Open source	0	0
Laptop computer	Already owned	0	0
Internet connection (data bundles)	9 weeks	5,000	45,000

Item	Quantity	Unit cost (FCFA)	Total (FCFA)
Hosting	Runs on the analyst's computer	0	0
Subtotal material and services			**45,000**
Familiarisation with the company and its products	3 days	25,000	75,000
Study of the security tools and manual assessment	7 days	25,000	175,000
Requirements and design of the scanner	5 days	25,000	125,000
Implementation (3 increments)	18 days	25,000	450,000
Tests, measurements and documentation	7 days	25,000	175,000
Subtotal handwork	40 days		**1,000,000**
TOTAL (material + handwork)			**1,045,000**

The handwork is valued at an indicative daily rate for a junior developer. Compared with a commercial scanner whose licence costs several million FCFA per year, the solution has no recurring cost.

4.8 Limits of the current version

- The built-in checks are passive: they detect configuration weaknesses, not injection flaws (SQL injection, XSS); those are left to OWASP ZAP.
- OWASP ZAP was used manually during the internship and its report format is parsed by the scanner, but the final version 2.1 was validated only with the external tools not configured; the ZAP, Nikto and Nmap stages still need to be run again on an authorized target with the new version.
- The URL risk score is a heuristic based on the text of the URL: it does not consult a reputation database and can be wrong in both directions.
- Scans of pages behind a login and scheduled scans are not supported.
- The tool is designed for one user on one machine; it has no user accounts.

Conclusion

The scanner gathers in one interface the checks that were previously done by hand with several tools, stores them, classifies them by OWASP category, scores them and exports them, while keeping the authorization at the centre of the workflow. It is validated by 21 automated tests and gives the expected grades to the two variants of the laboratory site.

GENERAL CONCLUSION

Summary of the work

This internship at IWOMI Technologies, from July 1 to August 31, 2026, started with the reproduction of the company website, which made me familiar with its image and products. The main work then answered a real problem observed during the internship: the security check of a website was slow, scattered among several tools, without score or history, and left the question of authorization to the memory of the analyst.

Methodology and solution

After studying the OWASP Top 10, the security controls of browsers and the existing tools, and after carrying out a manual assessment of an authorized website, I specified 17 functional and 8 non-functional requirements and built a **Web Application Vulnerability Scanner** in Python and Flask in three increments. The scanner crawls the target, checks headers, cookies, TLS, redirects and URL risk, can call ZAP, Nikto and Nmap, classifies its findings by OWASP category, computes an explainable score and grade, stores everything in SQLite and produces a PDF report. It is validated by 21 automated tests, all passed, on a laboratory website built in a weak and a hardened variant.

Difficulties encountered

The first difficulties were practical: configuring Python and the tools on Windows and correcting indentation errors. The main lesson came later: the tests revealed defects that manual use had not shown – a crashing history page, an `ftp://` target accepted, results lost after a restart, a label claiming that a site was “real”. Writing a laboratory site whose weaknesses were known in advance was the key to testing the scanner honestly and legally.

Perspectives

In the near future: run again the ZAP, Nikto and Nmap stages on an authorized target with version 2.1; add scans of pages behind a login; compare two scans of the same site to show the progress; enrich the URL analysis with a reputation service. In the longer term, the tool could be offered by IWOMI Technologies to its banking clients as a simple, regular “security health check” of their web interfaces, always within a written authorization.

Beyond the technical results, this internship taught me how a fintech works with banks, why security is part of the trust it sells, and how to lead a small project from an observed problem to a tested and documented tool. It confirmed my interest in software engineering and cybersecurity.

BIBLIOGRAPHY AND WEBOGRAPHY

Bibliography

1. OWASP Foundation (2021). *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*. OWASP.
2. Stuttard, D., & Pinto, M. (2011). *The Web Application Hacker's Handbook* (2nd ed.). Wiley.
3. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
4. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
5. Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC 8446). IETF.
6. Hodges, J., Jackson, C., & Barth, A. (2012). *HTTP Strict Transport Security (HSTS)* (RFC 6797). IETF.
7. Moriarty, K., & Farrell, S. (2021). *Deprecating TLS 1.0 and TLS 1.1* (RFC 8996). IETF.
8. Grinberg, M. (2018). *Flask Web Development* (2nd ed.). O'Reilly.
9. Republic of Cameroon (2010). Law No. 2010/012 of 21 December 2010 relating to cybersecurity and cybercriminality in Cameroon.
10. Republic of Cameroon (2024). Law No. 2024/017 of 23 December 2024 relating to the protection of personal data.

Webography

1. OWASP Top 10: https://owasp.org/Top10/
2. OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
3. OWASP ZAP documentation: https://www.zaproxy.org/docs/
4. Nikto: https://github.com/sullo/nikto · Nmap reference guide: https://nmap.org/book/man.html
5. MDN Web Docs – HTTP headers, HTML and CSS: https://developer.mozilla.org/
6. Flask documentation: https://flask.palletsprojects.com/ · Requests: https://requests.readthedocs.io/ · ReportLab: https://docs.reportlab.com/
7. We Are Tech Africa, "Cameroun : Collins Fomba Ken crée des outils numériques pour banques et consommateurs": https://www.wearetech.africa/
8. Business in Cameroon, "Cameroon: CBC opens first cash deposit ATM" (2018): https://www.businessincameroon.com/
9. WeeTracker, "Cameroon fintech IWOMI raises funding" (3 February 2025): https://weetracker.com/2025/02/03/cameroon-fintech-iwomi-raises-funding/

10. CB Insights, IWOMI Technologies company profile: https://www.cbinsights.com/company/iwomi-technologies

APPENDIX

Appendix 1: List of the automated tests

Table 18: *Automated tests of the scanner*

Area	Test	Result
Target validation	https:// added, http:// kept	Passed
Target validation	5 invalid inputs refused (empty, ftp://, no host, space, invalid characters)	Passed (5)
Crawler	Same-host pages and forms found	Passed
Crawler	Page limit respected	Passed
Headers and cookies	Weak site: missing headers and cookie flags	Passed
Headers and cookies	Hardened site: no missing header	Passed
Headers and cookies	Server header reported as information	Passed
Redirects and TLS	Long redirect chain detected	Passed
Redirects and TLS	HTTP target flagged	Passed
URL risk heuristic	Low / suspicious / high scores	Passed
URL risk heuristic	Low-risk label never claims a site is real	Passed
External tools	ZAP stage reported as skipped when not configured	Passed
Correlation and score	Findings deduplicated and sorted	Passed
Correlation and score	Score and grade	Passed
Web application	Scan refused without authorization	Passed
Web application	Full scan flow and persistence	Passed
Web application	History and URL check pages	Passed
Total	**21 tests**	**21 passed**

Appendix 2: Extract of the scan function

```
def perform_scan(scan_id, target):
    all_findings = []; stats = {'pages': 0, 'forms': 0, 'urls': 0}
    try:
        _update(scan_id, 'validating', 5, message='Validating target')
        ok, msg, normalized = validate_target(target)
        if not ok: raise ValueError(msg)
        _update(scan_id, 'discovering', 12, message='Crawling in-scope pages and forms')
        crawl_data = crawl(normalized, max_pages=40)
        save_pages(scan_id, crawl_data['pages'])
        stats = {'pages': len(crawl_data['pages']), 'forms': len(crawl_data['forms']),
                 'urls': crawl_data['discovered_urls']}
        _update(scan_id, 'browser', 30,
                message='Analyzing security headers, cookies and browser policies', **stats)
        all_findings += scan_browser_security(normalized)
        _update(scan_id, 'tls', 40, message='Checking TLS, redirects and URL risk', **stats)
        all_findings += (scan_tls(normalized) + scan_redirects(normalized)
                         + scan_phishing(normalized))
        _update(scan_id, 'zap', 50, message='OWASP ZAP stage', **stats)
        all_findings += run_zap(normalized)
        _update(scan_id, 'nikto', 68, message='Nikto stage', **stats)
        all_findings += run_nikto(normalized)
        _update(scan_id, 'nmap', 78, message='Nmap stage', **stats)
        all_findings += run_nmap(normalized)
        _update(scan_id, 'risk', 90, message='Correlating and de-duplicating findings', **stats)
        findings = normalize_findings(all_findings)
        save_findings(scan_id, findings)
        score, grade = risk_score(findings)
        _update(scan_id, 'complete', 100, message='Scan complete',
                findings=len(findings), score=score, grade=grade, **stats)
        update_scan(scan_id, finished_at=datetime.now().isoformat(sep=' ', timespec='seconds'))
    except Exception as e:
        save_findings(scan_id, normalize_findings(all_findings))   # keep what was found
        _update(scan_id, 'error', 100, message='Scan stopped because of an error',
                error=str(e), trace=traceback.format_exc(), **stats)
```

Appendix 3: Additional screenshots

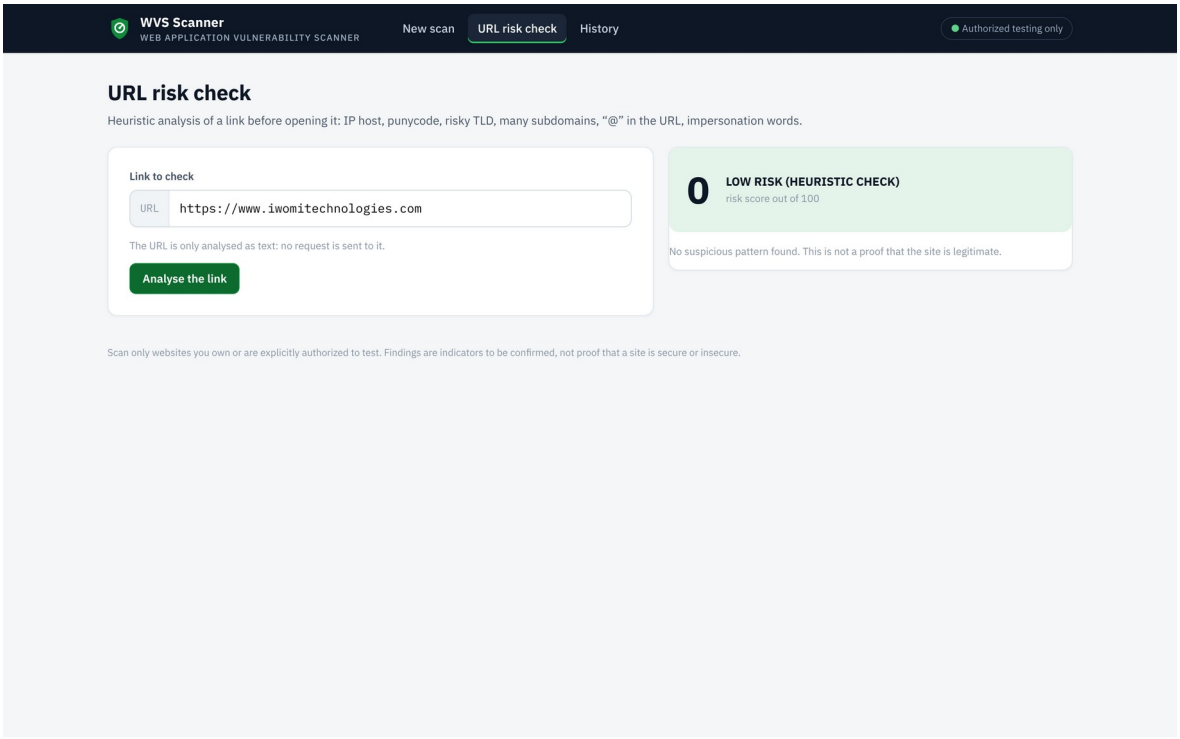


Figure 22: *URL risk check of the company domain: low risk (heuristic check) (source: author)*

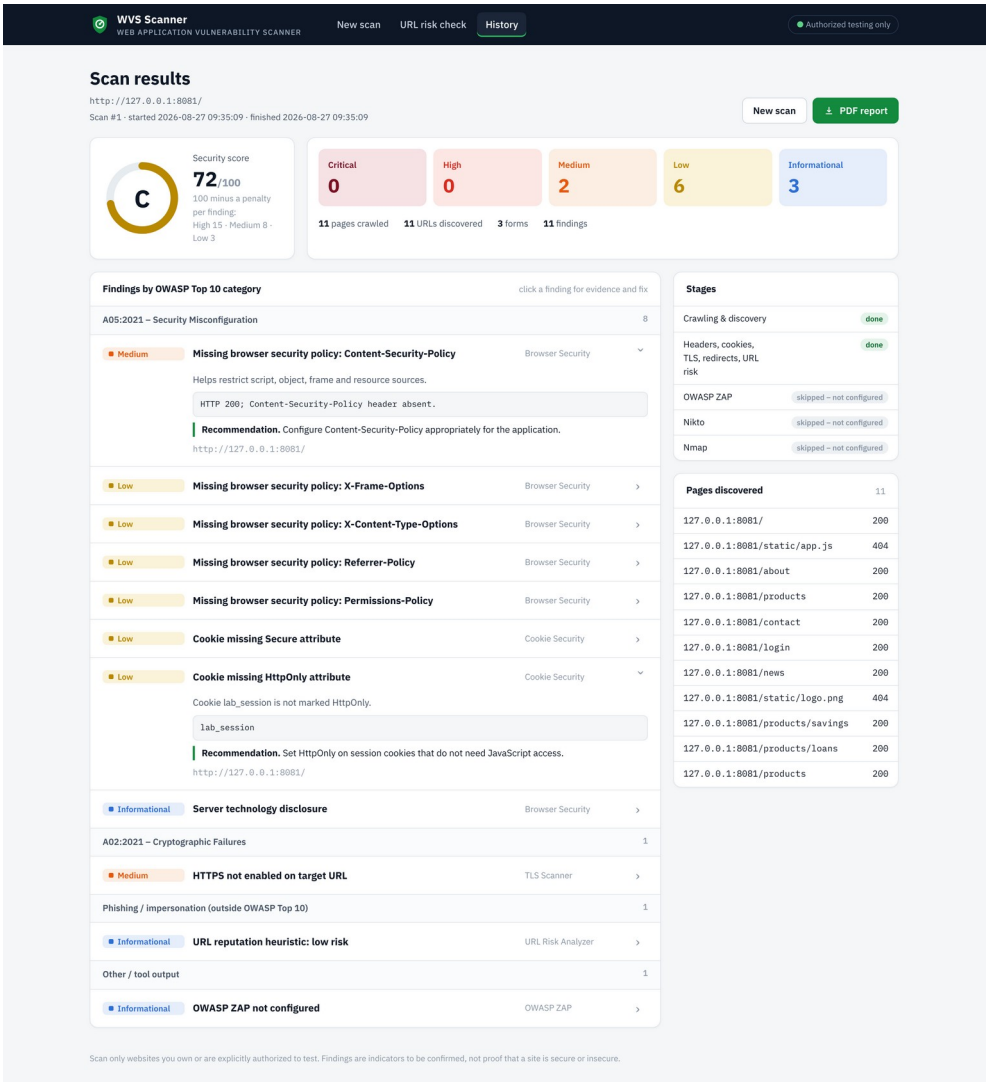


Figure 23: *Complete results page of the weak laboratory site (source: author)*